

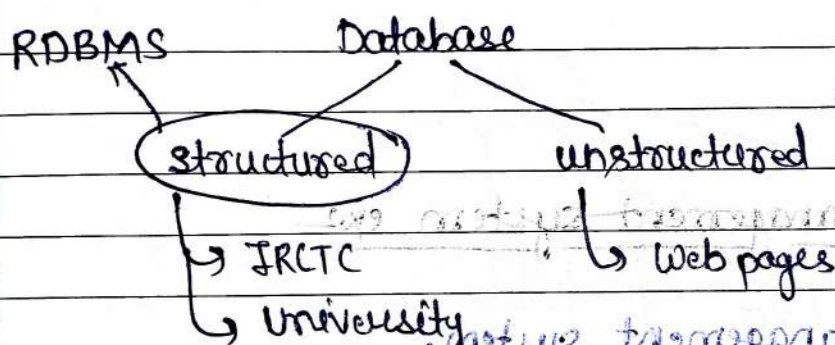
DBMS

① Data :-

① It is a raw facts and figures.
Ex- number, etc.

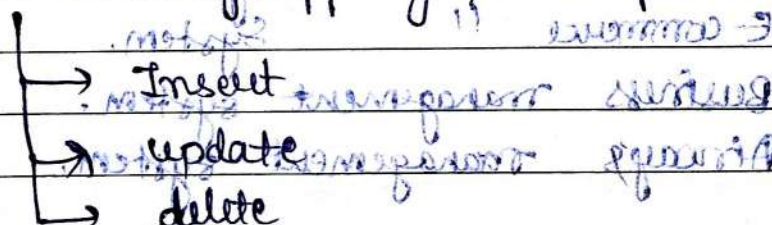
① Data base :-

① It is the collection of related data.



① Database management :-

① To manage a data by applying some operations



① Database management system :-

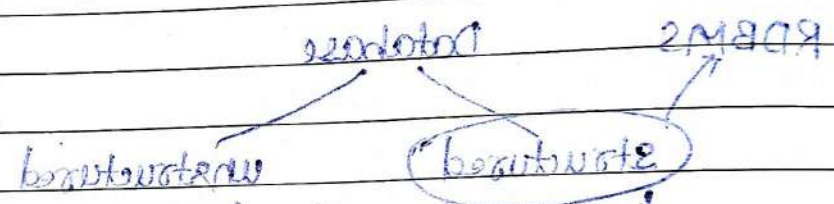
① Database management system.

① It is a set of programs that create, delete, update, insert, retrieve or select.

② Databases:-

- MySQL
- Mongo DB
- Oracle
- Redis
- IBM DB2
- SQLite
- Sybase
- RDBMS
- Fox-pro
- MS-Access

It is a collection of data files. It is used to store data.



③ Database management system ex:-

1. Student management system.
2. Hospital management system.
3. Library system.
4. Bank system.
5. Hotel management system.
6. E-commerce system.
7. Business management system.
8. Airways management system.

④ Difference between compiler and interpreter?

Compiler:-

⑤ A compiler translates the entire source code into

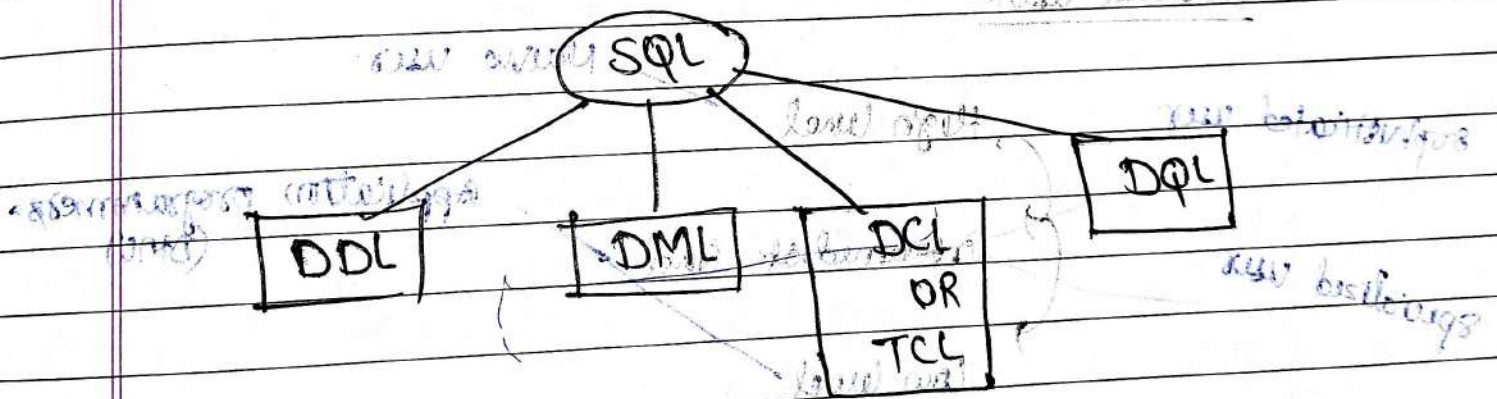
machine code before execution since no translation is needed during execution.

① Interpreter:-

② Interpreter translate code line by line during execution making it easier to select errors but slow the program.

③ SQL:-

④ Structured query language.



Q

Database vs file system - used data in the database millions growth between

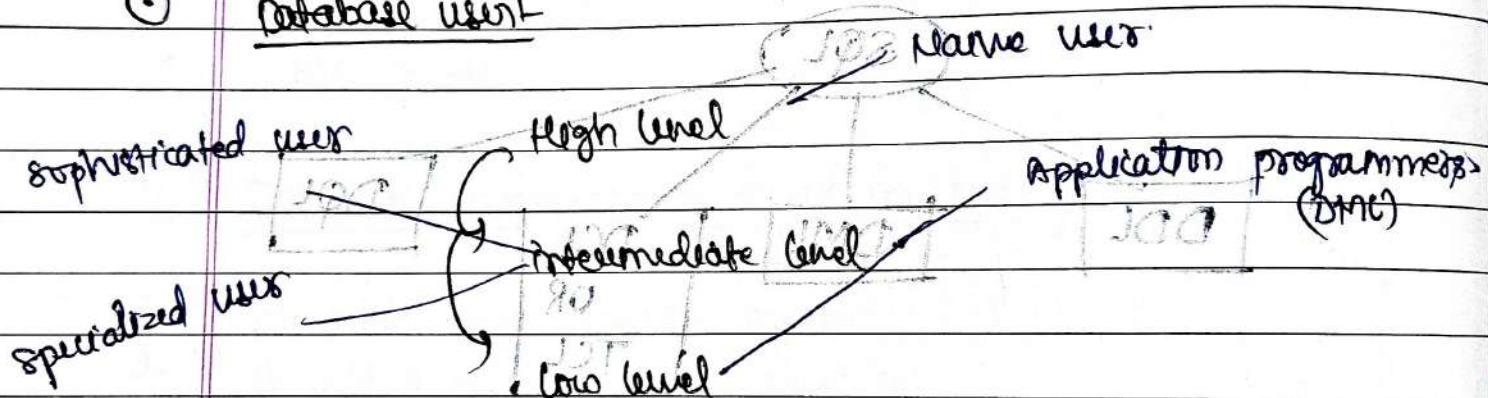
1. Data redundancy
2. Data inconsistency Inconsistence
3. Difficulty in access data.
4. Integrity problems
5. Atomicity problems
6. Concurrent access anomalies
7. Security problem
8. Data isolation.

Q

Database user and Administrator

Q

Database user



Date
29-01-24

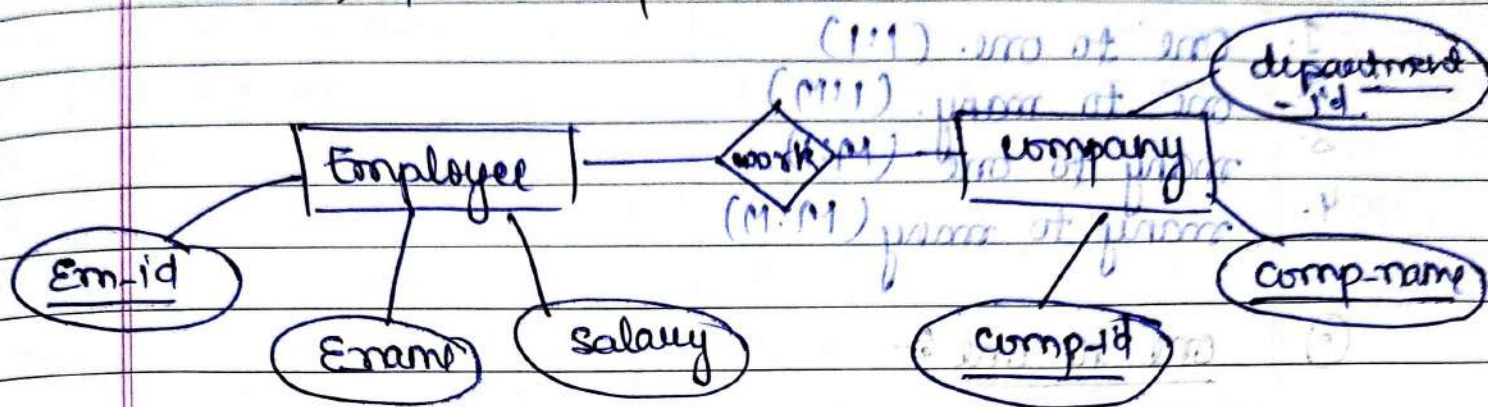
Date

Page No.

Shivatal

ER- Model :-

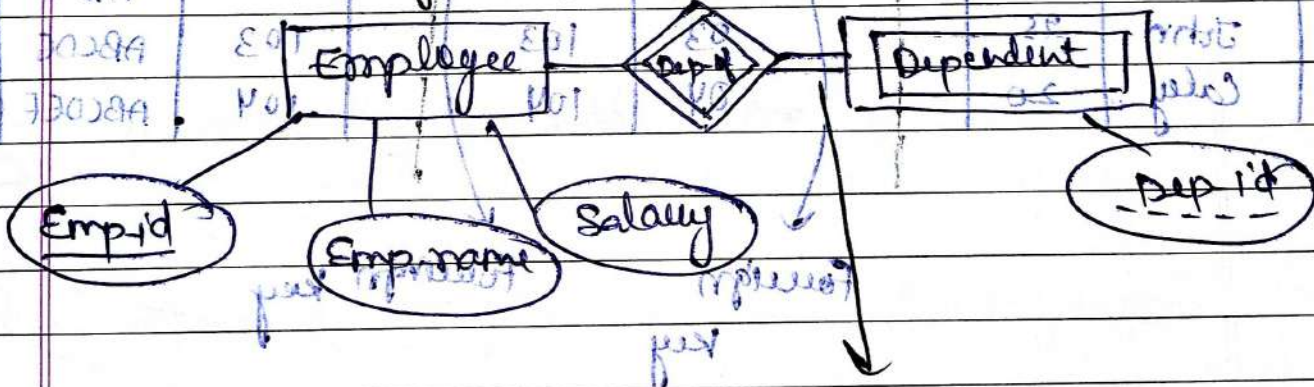
Relationship.



Entity



id	name	id	id	id	id	id	id	id
10	10	10	10	10	10	10	10	10
20	20	20	20	20	20	20	20	20
30	30	30	30	30	30	30	30	30
40	40	40	40	40	40	40	40	40



participation

total

partial

rebus prasa vya aro namo

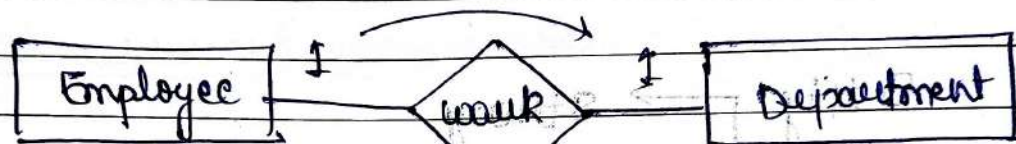
① Mapping constraints in DDL :-

② Degree of Relationship :-

Cardinality

1. One to one (1:1)
2. One to many (1:M)
3. Many to one (M:1)
4. Many to many (M:M)

③ one to one :-



<u>E-id</u>	E-name	E-age	<u>E-id</u>	D-id	<u>D-id</u>	D-name	D-loc
01	Raj	20	01	101	101	ABC	Aggra
02	David	21	02	102	102	ABCD	Delhi
03	John	25	03	103	103	ABCDE	Mumbai
04	Calay	20	04	104	104	ABCDEF	Mathura

Primary key

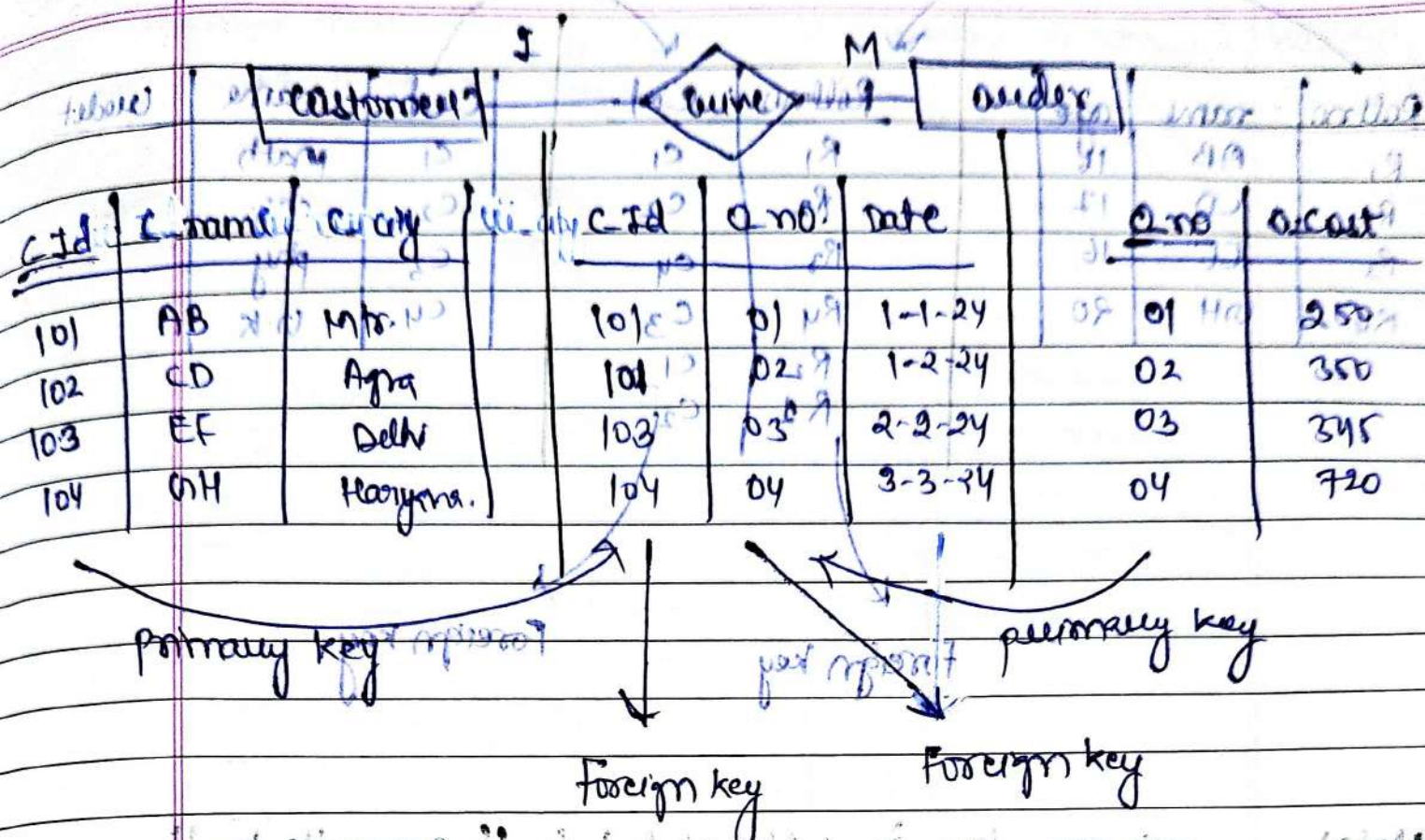
Foreign key

Foreign key

Note:- primary key in table 'work' is "Either E-id or D-id".
 → Reduce → 2.

④ one to many :-

⑤ customers can give many orders.

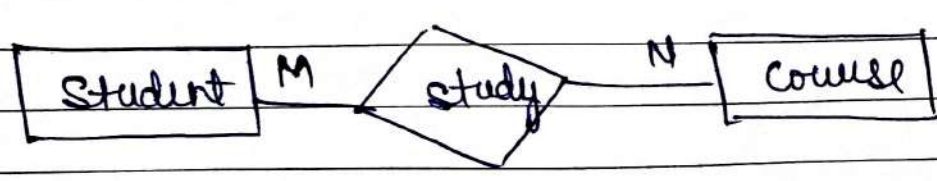


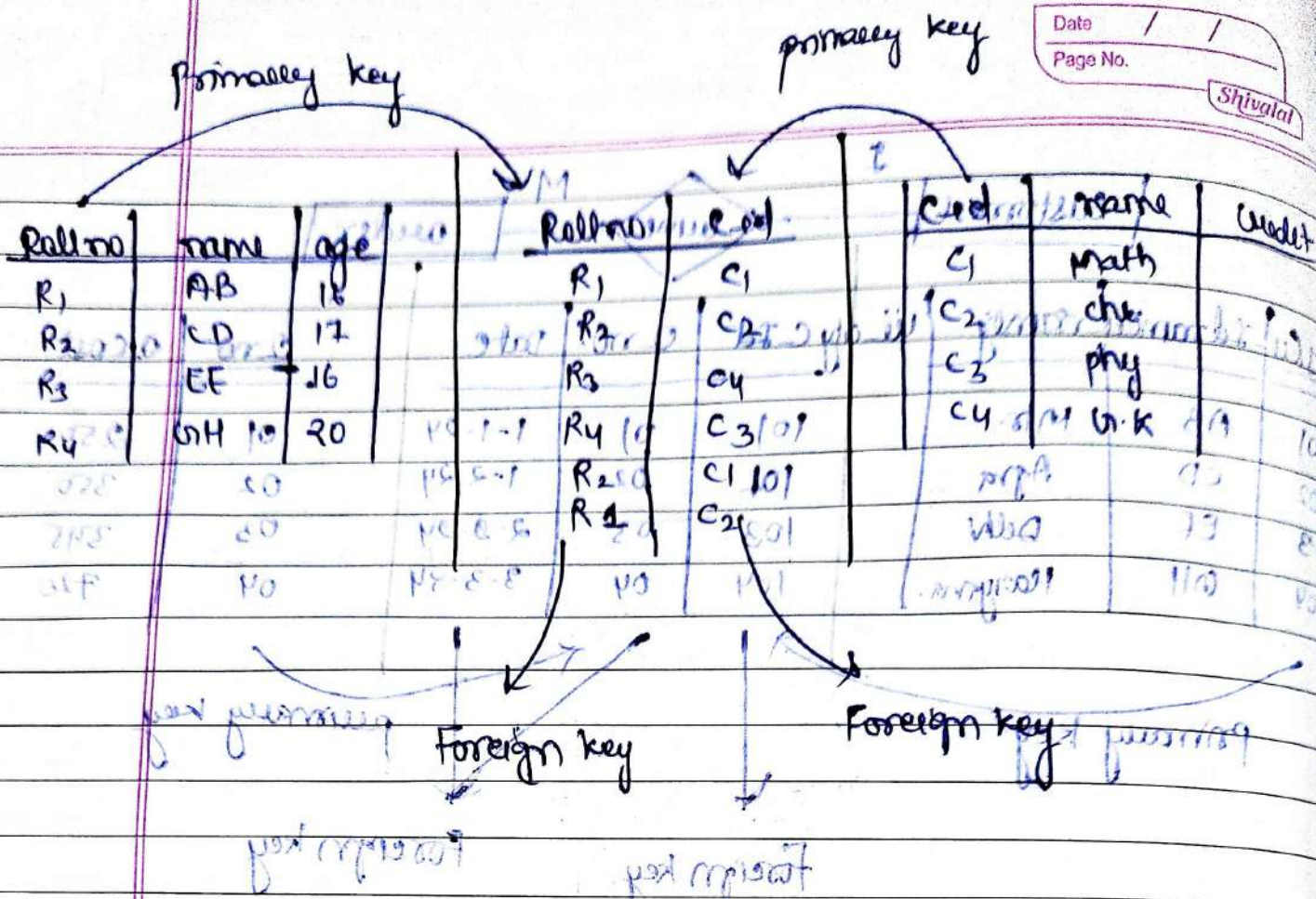
Note:- "The attribute of relationship is called Descriptive Attribute".

Note:- primary key in table 'buy' is "o_id".
 → reduce → 2

① Many to Many relation is 'no of pairs' ①

② denote with $M \rightarrow N$





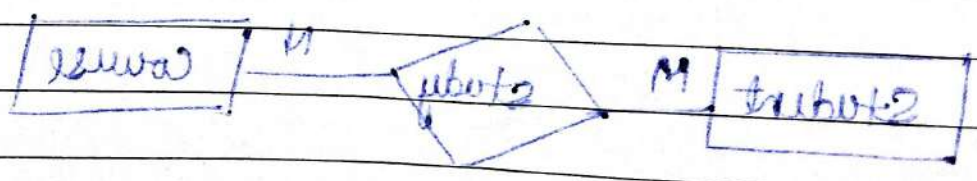
Note:- primary key in table 'study' is "composite key".

Composite key = Rollno, Cid
→ Reduce → 'NO' students

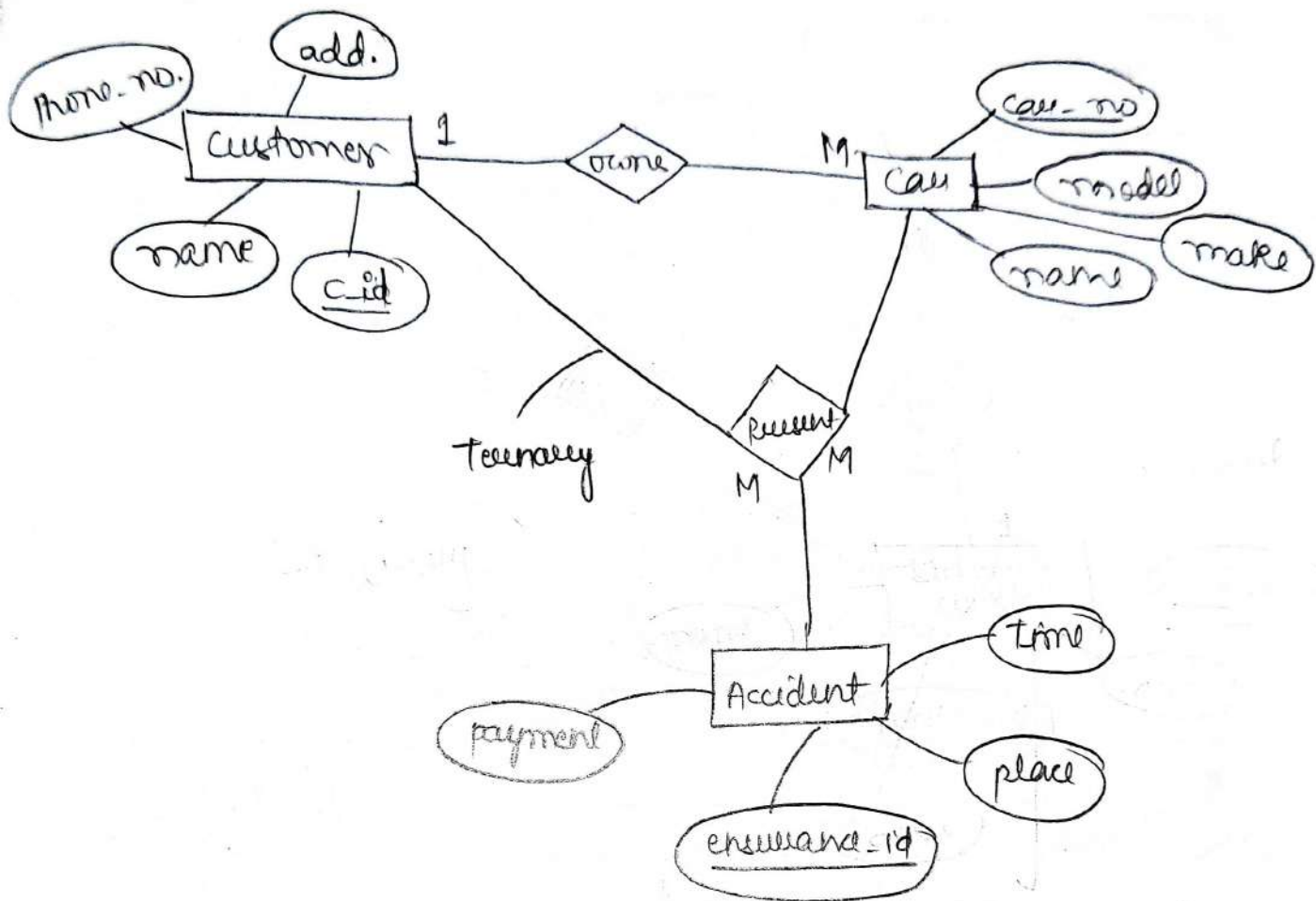
① "BI-C" is 'study' table is not primary key
many to one:-

② 'many to one' is just like 'one to many'

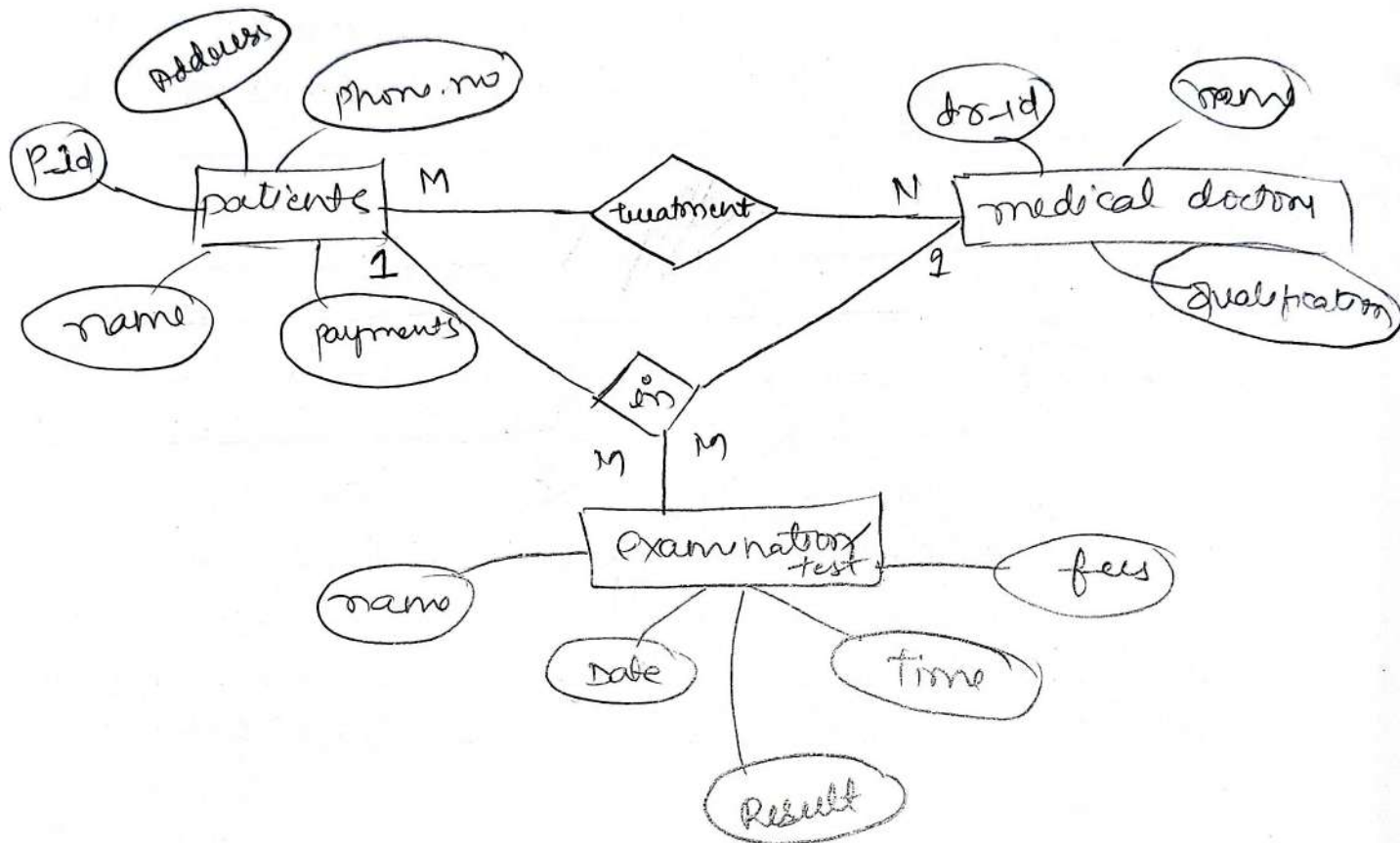
1 ← M few hands



1) Construct an ER-diagram for car insurance company whose customers own one or more cars each. Each car has associated with it zero to any no. of successful accident.

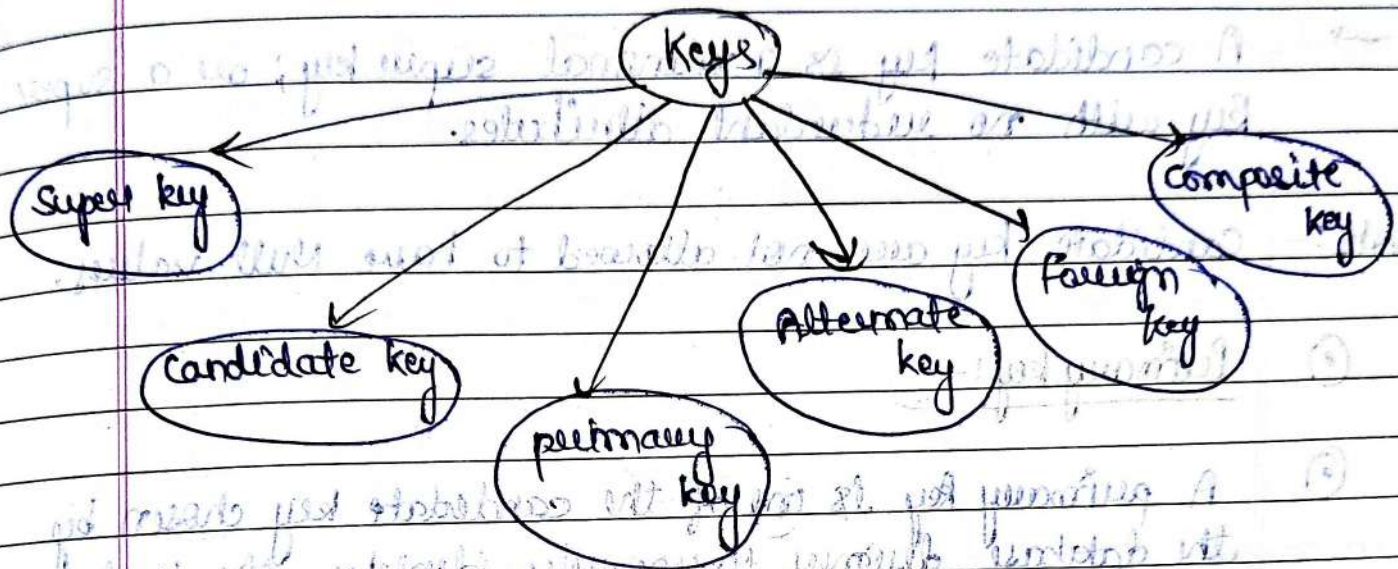


2. Construct an ER-diagram for a hospital with a set of patients and a set of medical doctors. Associated with each patient a log of the various tests and examination conducted,



① Keys in DBMS:-

① A key is an attribute or set of attributes that uniquely identifies any record (tuple) from the table.



1. Super key:-

① A super key is a combination of all possible attributes that can uniquely identify the records (or tuple) in the given relation.

→ It is a collection of maximum no. of attribute that uniquely identify the rows.

SK → Super key / CK → Candidate key / PK → Primary Key

Note:- $PK \subset CK \subset SK$

① All the possible set of keys that uniquely identify the tuple is known as Super key.

① Candidate key:-

② A candidate key is an attribute or set of attributes which can uniquely identify a tuple.

→ A candidate key is a minimal super key; or a super key with no redundant attributes.

Note:- Candidate key are not allowed to have Null values.

① Primary key:-

② A primary key is one of the candidate key chosen by the database designer to uniquely identify the tuple in the relation.

→ The value of primary key can never be null.

→ Always unique.

→ A relation is allowed to have only one primary key.

① Alternate key:-

②

$$PK \subset CK \subset AK$$

① Foreign key :-

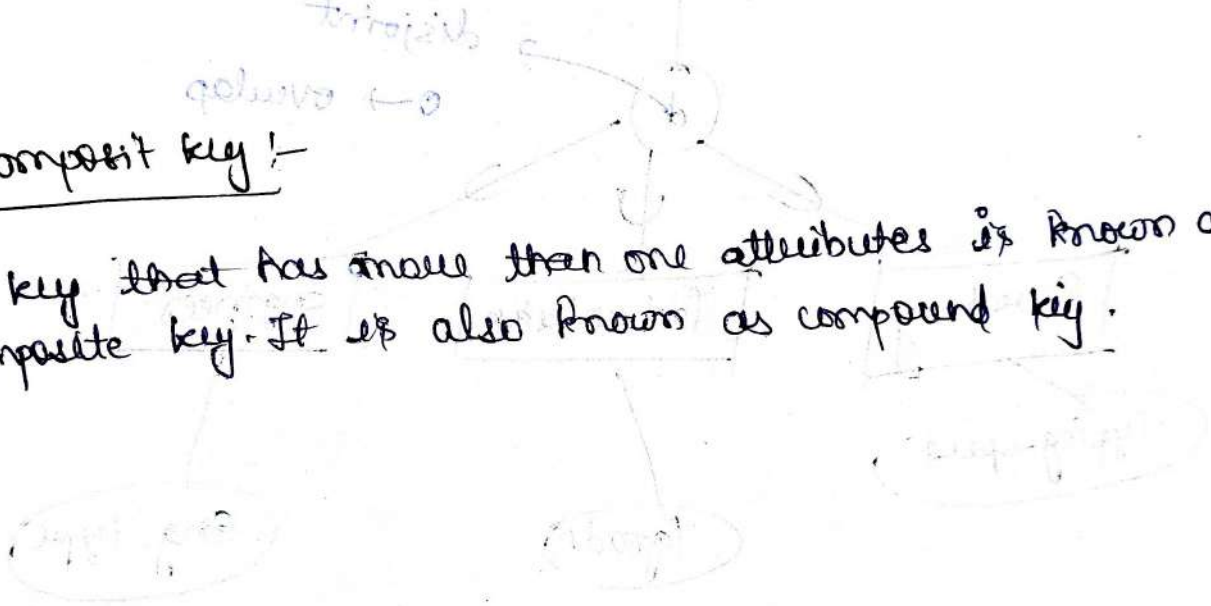
- A key used to link two tables together.
- An attribute (or set of attribute) in one table that refers to the primary key in another table.

② The purpose of the foreign key is to ensure (or maintain) referential integrity of the data.

- (i) Foreign key can have null values also.
- (ii) No restriction on a foreign key to be unique

③ Composite key :-

- ④ A key that has more than one attributes is known as composite key. It is also known as compound key.

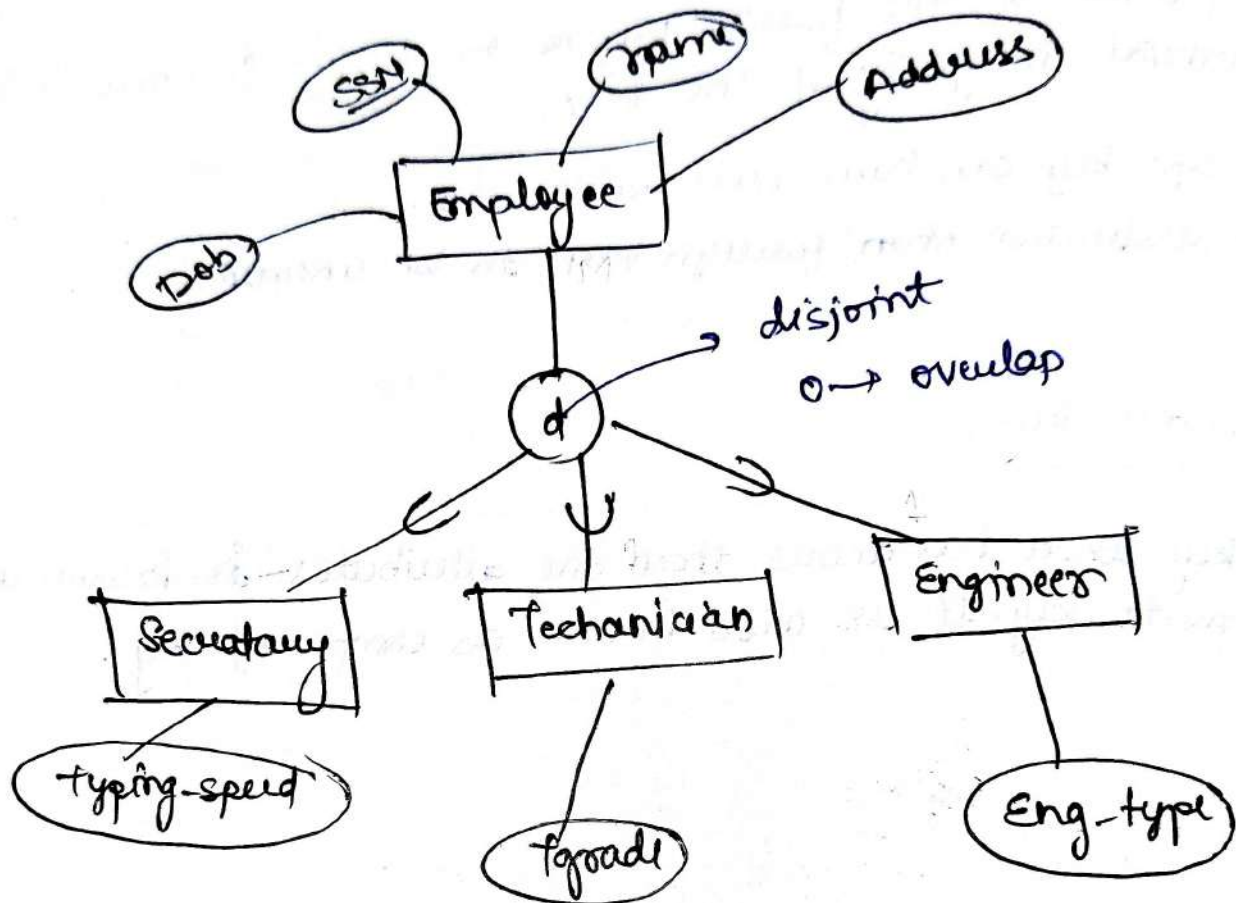


① BER :-

② Enhanced / Extended Entity relationship.

↳ Specialization.

↳ Generalization.



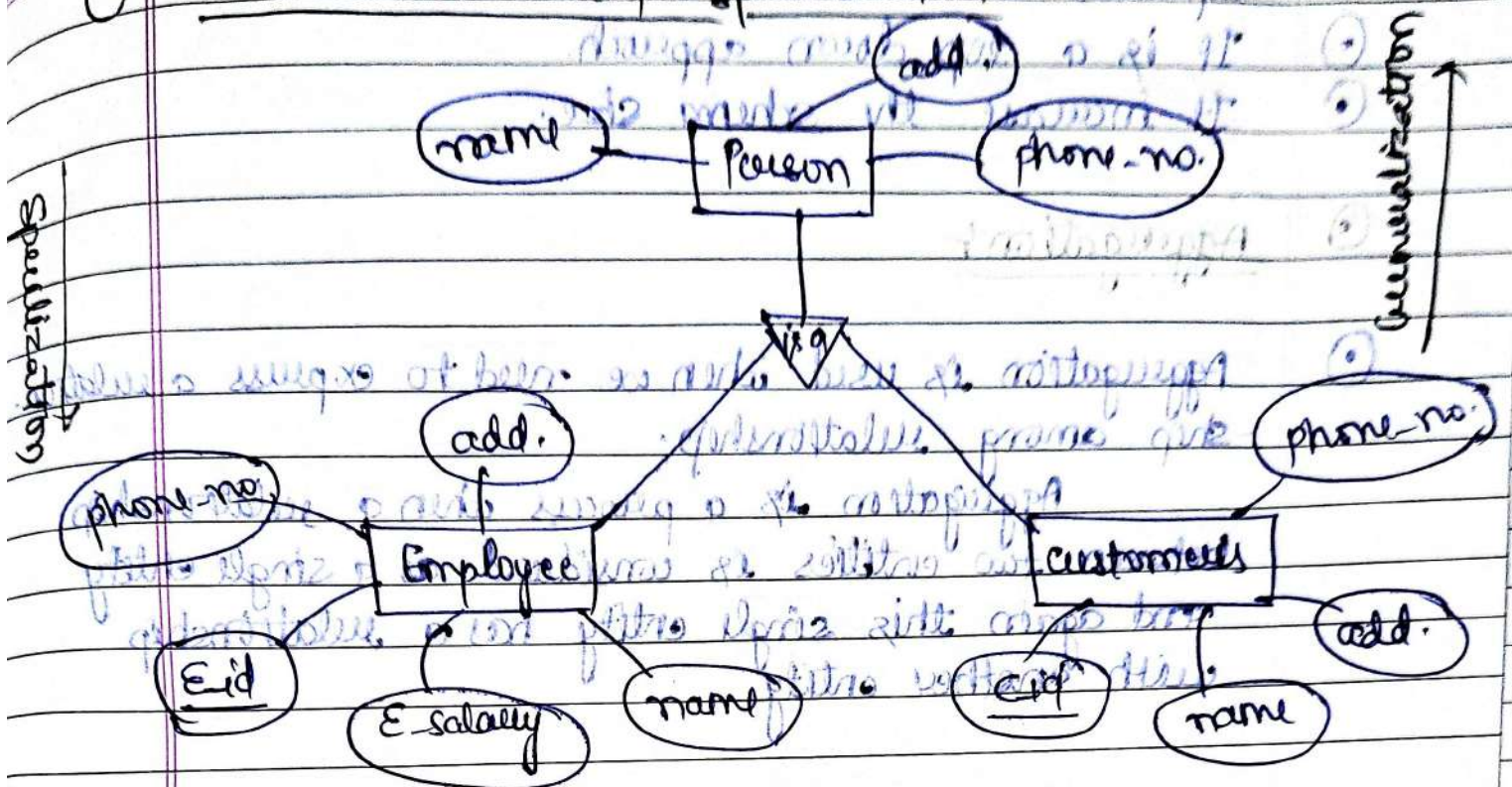
Employee-d (Dob , SSN , name , address)

Secretary (typing-speed , SSN)

Technician (Tgrade , SSN)

Engineers (Eng-type , SSN)

① Generalization and specialization (bottom up)



(i) Generalization (Bottom up)

- Employee (name, add., phone-no., Eid, E-salary)
- Customers (name, add., phone-no., Cid)
- Person (name, add., phone-no.)

#

Reduce the schema size.

①

It is the process of grouping lower level entities into broader categories (High level entity) based on common attribute.

②

It is a bottom up approach.

(ii)

Specialization (Top down)

③

It is the process of dividing a higher level entity into lower level entities based on uncommon

(Special) attribute

①

It is a top down approach.

②

It increase the schema size.

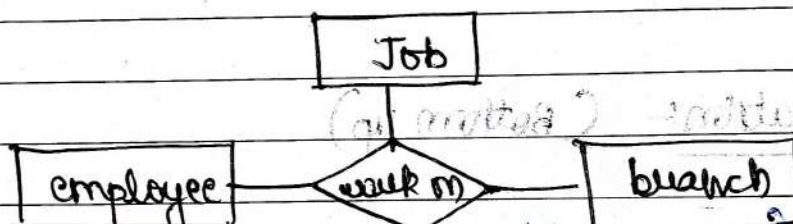
③

Aggregation

④

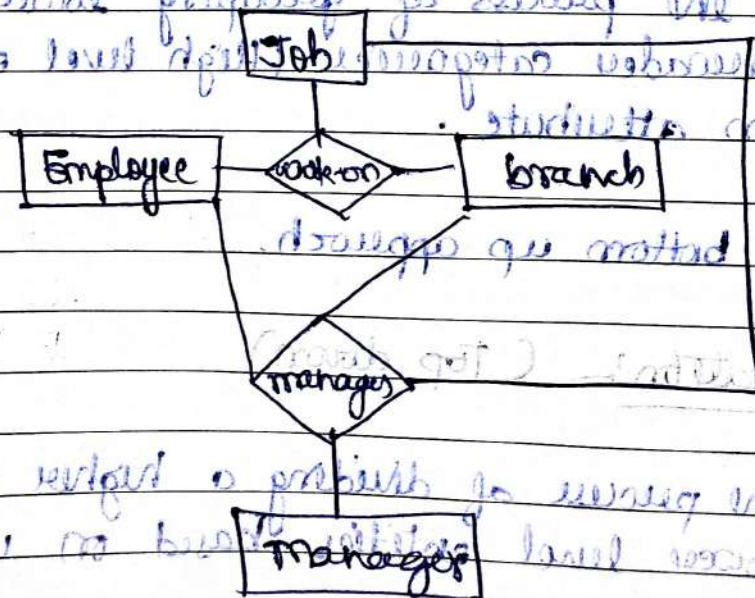
Aggregation is used when we need to express a relationship among relationship.

Aggregation is a process when a relationship between two entities is considered as a single entity and again this single entity has a relationship with another entity.



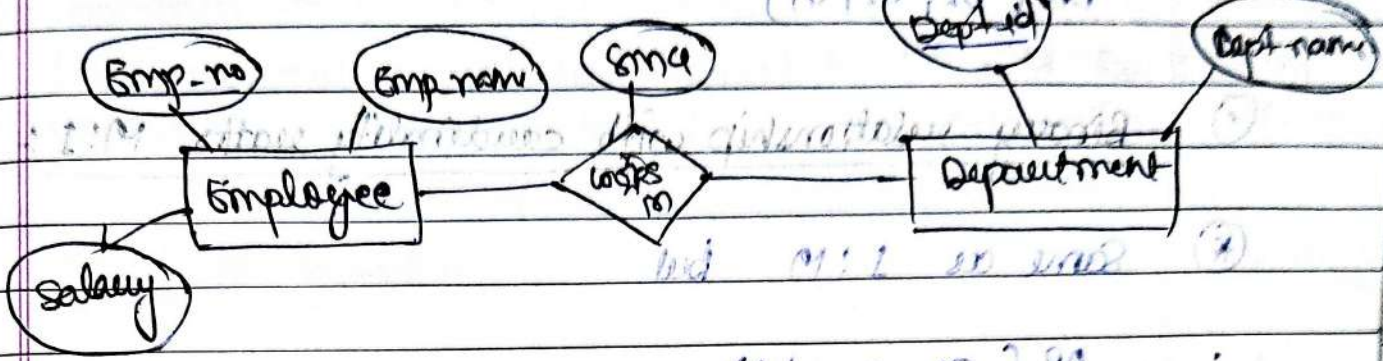
⑤

Suppose we want to assign a manager for jobs performed by an employee at a branch (i.e. want to assign managers to each employee, job, branch combination).



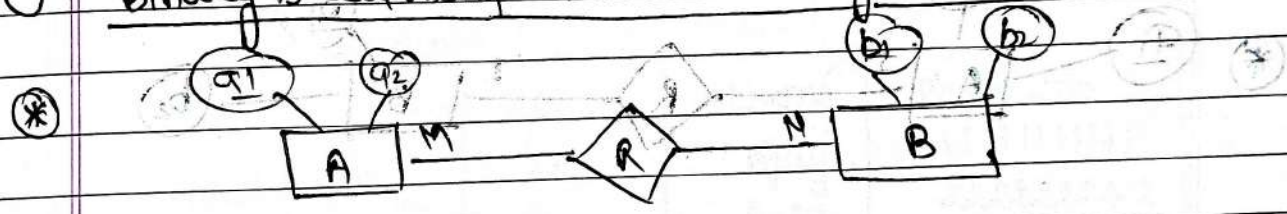
① ER to table reduction:

① relationship set:



Employee (Emp-no, Emp-name, salary)
 works in (sno, Emp-no, Dept-id)
 Department (Dept-id, Dept-name)

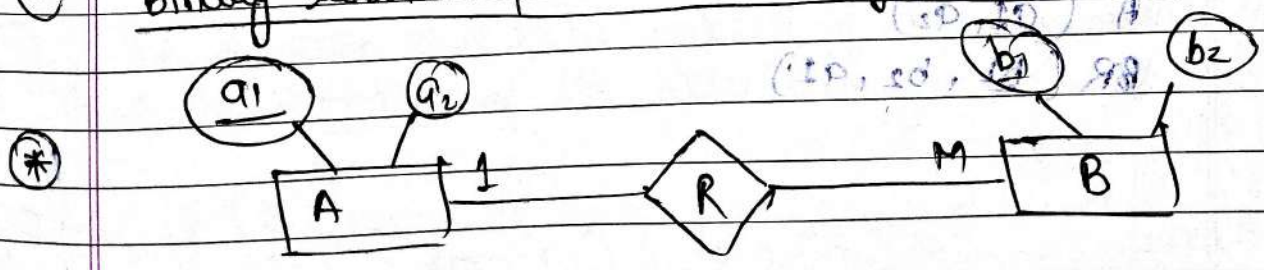
② Binary relationship with cardinality ratio 'm:m'



③ many to many, three table out:

A (a1, a2)
 R (a1, b1)
 B (b1, b2)

④ Binary relationship with cardinality ratio 1:m



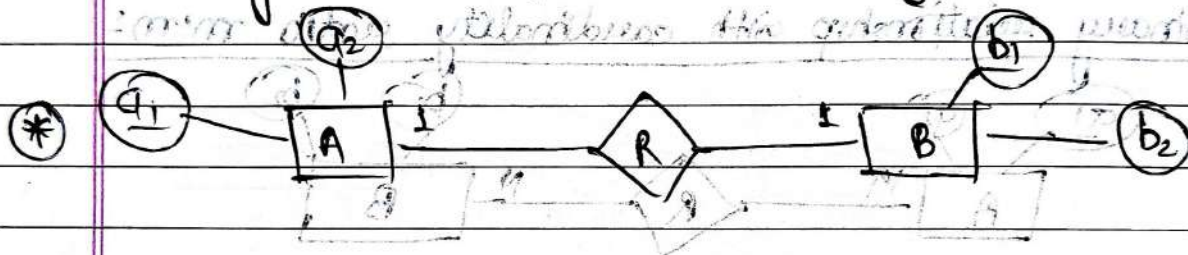
- $$A(\underline{a_1}, a_2)$$
- $$BR(\underline{b_1}, b_2, a_1)$$

- (*) Same as 2:19 but

$$AR(q_1, q_2, b_1)$$

B (bi (two) , graph (write)) graphical
(bi (two) , graph (write)) graphs
(bi (two) , graph (write)) graphs

- ② Binary relationship with cardinality ratio



- ① one to one, two table with primary & foreign

copy-1

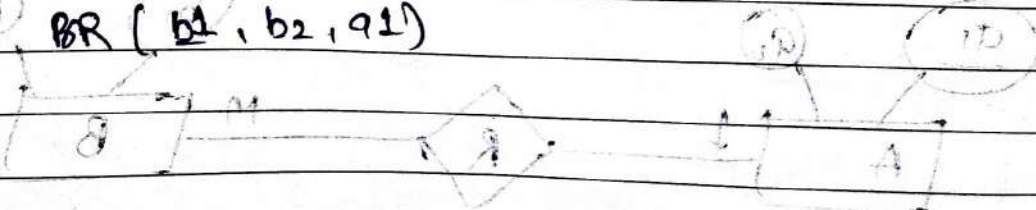
$$AR(a_1, a_2, b_1)$$
$$B(b_1, b_2)$$

(10, 10) A

(1d, 10, 2, 8)

(2010) 2

way-2

$$A(\underline{a_1}, a_2)$$
$$BR(b_1, b_2, q_1)$$


① Relational data model and language:-

② The relational model is the theoretical basis of relational database.

The relational model was proposed by E.F. Codd for IBM in 1970 to model data in the form of relations as table.

③ Relational model:-

④ Relational model can be represented as a table with columns and rows.

→ Each row is known as a tuple.

→ Each table of the column has a name or attribute.

Students

Rollno	Name	Phone no.
1.	Ajay	1111111111
2	Ram	2222222222
3	Aman	3343456781
4	Raj	5432101234

← row/tuple

← column/attribute

⑤ Relation:- A relation is a tuple with columns and rows.

⑥ Relation state:-

⑦ The relation state is a subset of the cartesian product of the domain of its attribute.

→ If $R(A_1, A_2, \dots, A_n)$
 $\mu(R) \subset \text{dom}(A_1) \times \text{dom}(A_2) \times \dots \times \text{dom}(A_n)$
 where, $\mu(R)$ is a specific state of relation R

① Integrity constraints :-

① Integrity constraints are rules defined in a database to maintain data accuracy, consistency and reliability.

They ensure the data stored in the database follows predefined rules or conditions.

Integrity constraints

Domain constraints

Entity Integrity constraints

Referential Integrity constraints

Key constraints

1. Domain constraints :-

① Domain constraints define the domain or the valid set of values for an attribute.

② The data type of domain includes string, characters, int, time, date, currency etc.

2. Entity integrity constraint :-

① The entity integrity constraint states that primary key value can't be null.
 $\& [PK] \neq \text{Null}$

This is because the primary key values is used to identify individuals even in relation and if the primary key has a null value it will be impossible to identify the individual.

a null value, then we can't identify those rows.

3. Referential integrity constraints:-

- ① A referential integrity constraint is specified between two tables. We can't take those value which are not in (P.T) Referential Integrity constraint is enforced when a foreign key references the primary key of a table.
- Parent table

① Key constraints:-

- ① An entity set can have multiple keys or candidate keys (minimal superkey), but out of which one key will be the primary key.

→ Key constraints specified that in any relation:

- (i) All the values of primary key must be unique.
 - (ii) The value of primary key must not be null.
- artificial or surrogate key.

(Informal terms)	(Formal terms)
Table	Relation
Column header	Attribute
All possible column values	Domain
Row	Tuple
Table of definitions	Schema of a relation
Populated table	State of the relation

① Relational Algebra:-

① Relational Algebra is a 'procedural query language' which takes a relation as an input and generates a relation as an output.

→ Relational Algebra is a language for expressing relational database queries.

It uses operators to perform queries. An operator can be either unary or binary.

② Types of operators in relational algebra:-

(i) Basic / Fundamental operators.

(ii) Additional / Derived operators.

③ Basic operators

- ↳ Projection (π)
- ↳ Selection (σ)
- ↳ Cross product ($\times$)
- ↳ Union ($\cup$)
- ↳ Rename (ρ)
- ↳ Set difference ($-$)

④ Derived operators

- ↳ Join ($\bowtie$)
- ↳ Intersection ($\cap$)
- ↳ Division ($/, \div$)

(i) Projection:- (π)

⑤ It is a unary operator in relational algebra that performs a projection operation.

It projects the particular columns from a relation.

⑥ projection (π) corresponds to the select clause of a SQL command.

And = $\wedge$
Or = $\vee$

→ (Four column)

Syntax: π attribute list (relation name) table name)

Note: Duplicate rows are automatically eliminated from result.

(ii) Selection (σ) :-

① It is a unary operator in relational algebra that performs a selection operation.

→ (For rows)

① It selects (rows) that satisfy the given condition from a relation.

Syntax: σ (condition) (Relation Name)

① selection(σ) corresponds to the where clause of a SQL query.

(iii) Cartesian product ($\times$) :- [Cartesian product]

① Cartesian product is fundamental operator in relational algebra. It combines information of two different relations into one.

① SQL: SELECT * FROM R₁, R₂

Syntax: Table-1 $\times$ Table-2

Ex: There are two table :- Table-1 and Table-2

→ Table-1 X Table-2

no. of Columns = Table 1 - column + Table 2 column
no. of Rows = Table 1 - rows X Table 2 - rows

Such as:

	A_1	A_2	$A_1 \times A_2$
Attributes	a	b	$a + b$
Rows / tuple	n_1	n_2	$n_1 \times n_2$

Notes:-

If both input relations have some attribute having same name, change the name of the attribute with the name of the relation "relation-name / table-name . attribute-name"

(iv) Union:-

- ⊙ Suppose R and S are two relations, The union operation selects all the tuples that are "either in relation R or S or in both relations R and S"

Syntax:-

R ∪ S

Note:- $A \cup B = B \cup A$

Note:-

Number of attribute should be same:

Union is commutative $R \cup S = S \cup R$

(v) Rename (ρ):-

- ⊙ The rename operator is used to rename the output of a relations.

Ex- Relation name 'Employee' change name to Emp

(i) $\rho_{Emp}(Employee)$

(ii) $\rho_{Emp}(-, -, -)(Employee)$

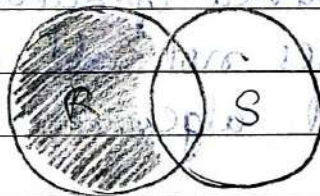
(vi) Set difference $\leftarrow$

① Suppose R and S are two relations. The set Difference operation selects all the tuples that are present in first relation R but not in second relation S.

Note Two relations R and S both have same number of attributes.
Corresponding attribute (or column) have the same domain (or type).

Syntax $R - S$

SQL:- $((SELECT * FROM R) EXCEPT (SELECT * FROM S));$



$R - S$

Note Set Difference is non-commutative: $(A - B \neq B - A)$

$\therefore R - (R - S) = R \cap S$

① Derived operators

(i) Intersect :-

① Suppose R and S are two relations. The set intersecting operation selects all the tuples that are in both relation R & S.

Note:- Both have same number of attributes.
Corresponding attribute (or column) have the same domain

Syntax:-

R ∩ S

SQL:-

(SELECT * FROM R)

INTERSECT

(SELECT * FROM S);

Note:-

Set Intersect is commutative $A \cap B = B \cap A$.

(ii) JOIN :-

① Join is an derived operator which simplify the queries, but does not add any new power to the basic relational algebra.

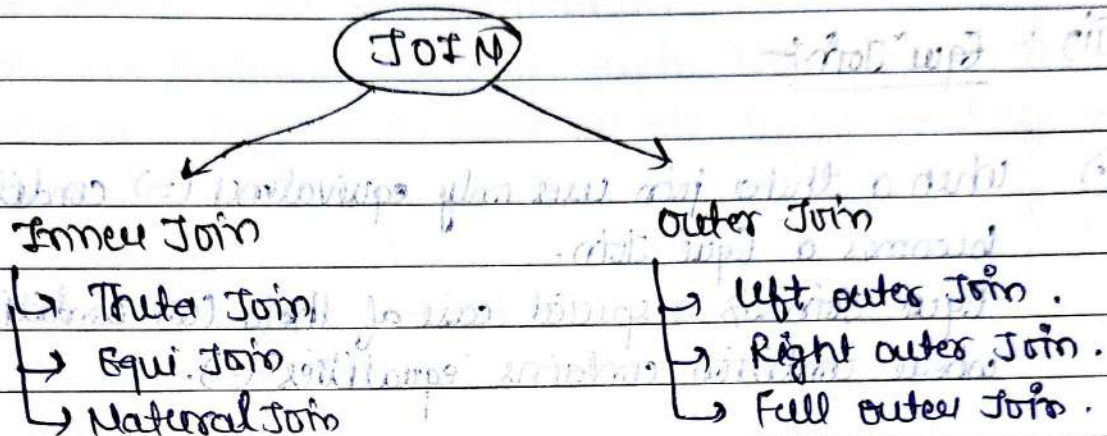
→ Join is a combination of a cartesian product followed by a selection process.

(A-R) (B-A) Join is cartesian product + selection.

② A Join operation pairs two tuples from different

relations, if and only if a given join condition is satisfied.

① Types of Join:-



① Inner join:-

① An Inner join includes only those tuples that satisfy the matching criteria, while the rest of tuples are excluded.

(i) Theta Join:-

① Also called conditional join.

→ It combines tuples from different relations provided they satisfy the θ condition.

→ It is a general case of join and it is used when we want to join two or more relation based on some condition.

The join condition is denoted by symbol θ .
It use all kinds of conditional operators like $<$, $>$, $<=$, $>=$, $=$, $\neq$

Syntax:-

$$A \bowtie_{\theta} B$$

where, θ = condition

Note:-

$$A \bowtie_{\theta} B = \sigma_{\theta} (A \times B)$$

(ii)

Equi Join:-

(M107)

- ① When a theta join uses only equivalence ($=$) condition, it becomes a Equi Join.

Equi Join is a special case of theta (or conditional) join where condition contains equalities ($=$).

Syntax:-

$$A \bowtie A.a_1 = B.a_1 B$$

(iii)

Natural Join:-

- ① Natural Join can only be performed if there is at least one common attribute (column) that exist between two relations. In addition, the attributes must have the same name and domain.

→

Natural Join does not use any comparison operator.

It is same as equi join which executes implicitly by comparing all the common attributes (columns) in both relation, but difference is that in Natural Join the common attributes appears only once.

Syntax:-

$$A \bowtie B$$

$$A * B \quad (\text{Najate})$$

Natural join = Cartesian product + selection + projection.

① outer join:-

① The outer join operation is an extension of the join operation that avoids loss of information.

→ outer join contains matching tuples that satisfy the matching condition, along with some or all tuples that do not satisfy the matching condition.

• It is based on both matched or unmatched tuple.

→ It uses null values.

outer join = Natural join + Extra information.

(from left table, right on both)

(i) left outer join:- (L)

① In left outer join, all the tuples from the left relation R_1 are included in the resulting relation. The tuples of R_1 which do not satisfy join condition will have values as Null for attributes of R_2 .

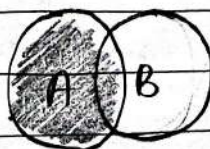
→ In short:-

• All records from left table.

• only matching records from right table.

Syntax:-

$R_1 \bowtie R_2$



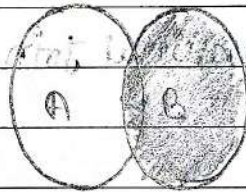
(ii) Right outer join :- (X)

- ① In right outer join, all the tuples from the right relation R_2 are included in the resulting relation. The tuples of R_2 which do not satisfy join condition will have value as Null for attribute of R_1 .

→ In short:-

- All records from right table.
- Only matching records from left table.

Syntax:- $R_1 \text{ X } R_2$

(iii) Full join :- (X)

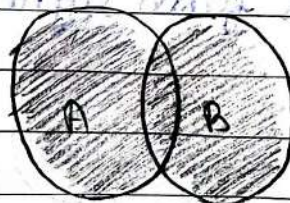
(X) :- join of both table (1)

- ① In Full join, all the tuples from both left relation R_1 and right relation R_2 are in resulting relation.

• In short:-

→ All records from all tables.

Syntax:- $R_1 \text{ X } R_2$



① Division:- is a derived operator. (2)

② Division operator is a derived operator, not supported as a primitive operator.

→ Situated to queries that includes the keywords all, every, at all, for all, in all, at every, for every, or in every etc.

Syntax:- $A \div B$ OR A/B

Ex- A B $A \div B$ OR A/B

x	y
a	1
b	2
a	2
b	4

x
1
2

x
a

Division operator can be applied if and only if.

- Attributes of B is proper subset of Attribute of A.
- The relation returned by division operator will have attribute = (A attribute of A - All attribute of B)

Note:- Division can be expressed in terms of cross product, set-difference and Projection.

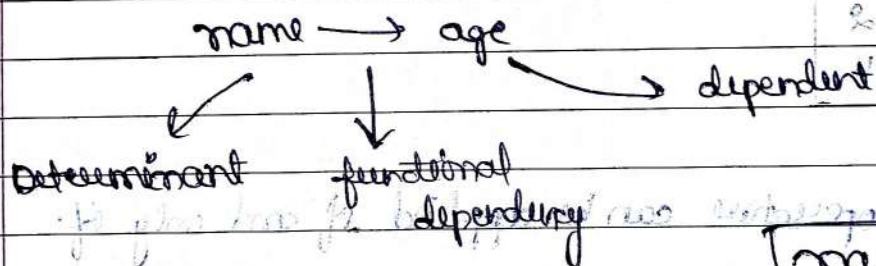
① Database Design & Normalization:-

① functional dependencies:-

① A functional dependency is a relationship between attributes of table dependent on each other.

Ex:- student

name	age
A	10
B	12
C	14
A	10



name → name

Functional dependencies

trivial when

ACY

Feeling

Partially

① A set of attributes X functionally determines a set of attributes Y if the values of X determine a unique value of Y .

2-34

① Inference rules for Functional Dependencies:-

(i) Reflexive:-

$$X \longrightarrow X$$

$$\forall Y, Y \subset X.$$

(ii) Augmentation:-

$$X \longrightarrow Y$$

$$\text{then, } XZ \longrightarrow YZ$$

(iii) Transitive:-

$$X \longrightarrow Y, Y \longrightarrow Z$$

then

$$X \longrightarrow Z$$

↑ upper

These all three are known as Armstrong axioms.

↓ lower

(iv) Decomposition:-

$$X \longrightarrow YZ$$

$$\boxed{Y \subset Z}$$

Then,

$$X \rightarrow Y$$

$$X \rightarrow Z$$

(v) union:-

$$X \rightarrow Y \text{ and } X \rightarrow Z$$

so,

$$X \rightarrow YZ$$

(vi) Pseudotransitivity:-

$$X \rightarrow Y \text{ and } WY \rightarrow Z$$

so,

$$WX \rightarrow Z$$

⑦ closure of fd:- (F^+)

If relation $R(A, B, C)$

$$F = \{ A \rightarrow B, B \rightarrow C \}$$

$$F^+ = \{ A \rightarrow B, B \rightarrow C, A \rightarrow C \} \text{ (}\therefore \text{ } R_2 \text{ trans.)}$$

$$\boxed{F \subset F^+}$$

$$\boxed{A \rightarrow Y}$$

① Normalization:-

② It is a technique to remove or reduce redundancy from a table.

(i) Insertion Anomaly.

(ii) Deletion Anomaly.

(iii) Update Anomaly.

③ It is a step by step process to remove redundancy and make our design (bad to good).

④ Steps are:-

1. 1NF

2. 2NF

3. 3NF

4. BCNF

5. 4NF

6. 5NF

Ex-

Roll no	Name	Course	Branch	HOD name	HOD Phone
1001	Abhi	B.Tech	CSE	XA	9999999999
2001	XYZ	B.Tech	EE	NP	123456789

1. Insertion redundancy of HOD information.

2. Delete the name XYZ of HOD information is also delete

3. To update the HOD name we have to update it thousands of time.

① Normal forms:-

① First normal form (1NF):-

② step 1 of the normalization process

③ There are 4 basic rules that a table should follow to be in 1st Normal form.

Rule 1:- Each column should contain atomic values.

Ex-

A	B
A1	B1
A2	B2, B3
A3	B4

X

Rule 2:-

Rule 3:-

A column should contain values that are of the same type.

Same names leads to confusion at the time of data retrieval.

Table name

Table name	DOB	Name	Name
X-X-Z	A	P	
X-Y-Z	B	Q	
A-B-C	C	R	

Rule 4:-

Order in which data is saved doesn't matter.

②

Second normal form (2NF):-

③

It should be in 1st normal form.

① And it should not have any partial dependencies.

Ques: Let $R(A, B, C, D, E, F, G)$
 $F = \{ A \rightarrow BC, BC \rightarrow DE, D \rightarrow F, CF \rightarrow G \}$

① 1NF:-

[2 | 8 | 9 | 1]

A relation R is in 1NF if and only if the domain of each attribute contains only atomic or (indivisible) values and the values of each attribute contain only a single value from that domain.

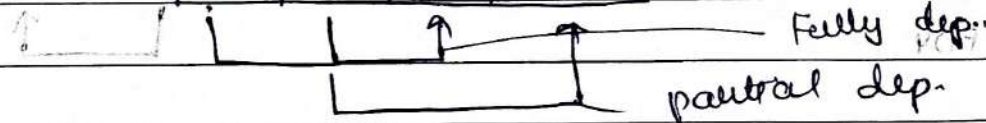
Ex- $R(A, B, C, D)$
 MV
 $A = 10.7$
 $B, C = 10.4$
 $\hookrightarrow$ is not in 1NF.

② 2NF:-

A relation schema R is in second normal form (2NF) if every non-prime attribute A in R is fully functionally dependent on the primary key.

Ex- $FD = \{ AB \rightarrow C, B \rightarrow D \}$ $R(A, B, C, D)$

R	A	B	C	D
---	---	---	---	---



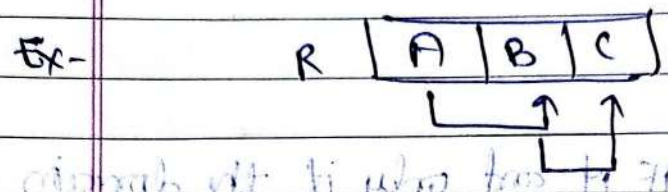
Candidate key = ? (AB)

prime attr = ? (AB)

non-prime = ? (C, D)

Q. 3NF -

A relation schema R is in third normal form (3NF) if it is in 2NF and no non prime attribute A in R is transitive dependent on the primary key.



does it satisfy 3NF? No, because A is a candidate key and B, C are non-prime attributes. $A \rightarrow B$ and $A \rightarrow C$ are functional dependencies. $(A)^+ = ABC = R$. $(B)^+ = BC \neq R$. Hence, it is not in 3NF.

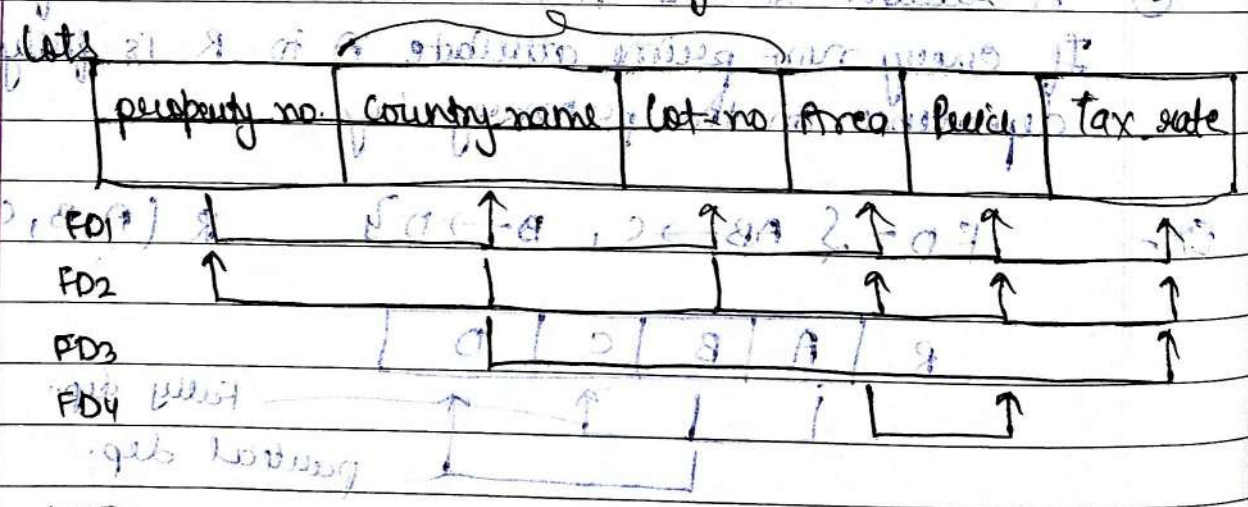
P.A = A
 N.P.A = B, C
 (A, B, C) / A
 VCI
 3NF or not?

For 3NF,

Non prime $\nrightarrow$ Non prime

(3NF) candidate key

Ques



(i) 1NF

$\rightarrow$ No change
 The given relation, there is no multivalued attribute.

(ii) 2NF

- Candidate keys: (country name, lot no, property no) -
- prime attr = (// //)
- Non prime attr = (// //) (Area, Price, Tax rate)

• country name $\rightarrow$ tax rate that is partial dependency

Then,

lot 1	Property id	Country name	lot #	Area	price
lot 2	country-name, Tax rate				

→ lot 1A

→ lot 1B

→ lot 2

- Transaction system:-

- A transaction is a collection of operations that perform a single logical function in a database application

→ Each transaction is a unit of both atomicity and consistency

- What if system fails?
- What if more than one user is concurrently updating the same data?

EX let's say your account is A and your friend's account is B and you are transferring 10000 from A to B, the set of operations are the steps of the transaction are:

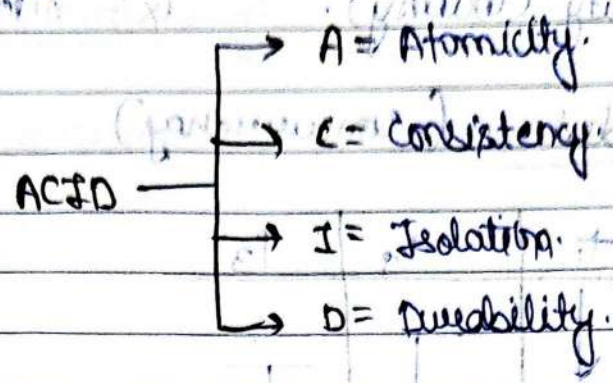
1. Read your account balance $R(A);$
2. Deduct the amount from your balance $A = A - 10000;$
3. write the remaining balance to your account $W(A);$
4. Read your friend's account balance. $R(B);$
5. Add the amount to his account balance $B = B + 10000;$
6. write the new updated balance to his account $W(B)$

① This whole set of operations can be called a transaction where,

R is Read and W is write.

- ② Transaction - management component.
- ③ concurrency- control manager.

In order to maintain consistency in a database, before and after the transaction, certain properties are followed this is called ACID properties.



① Serializability:-

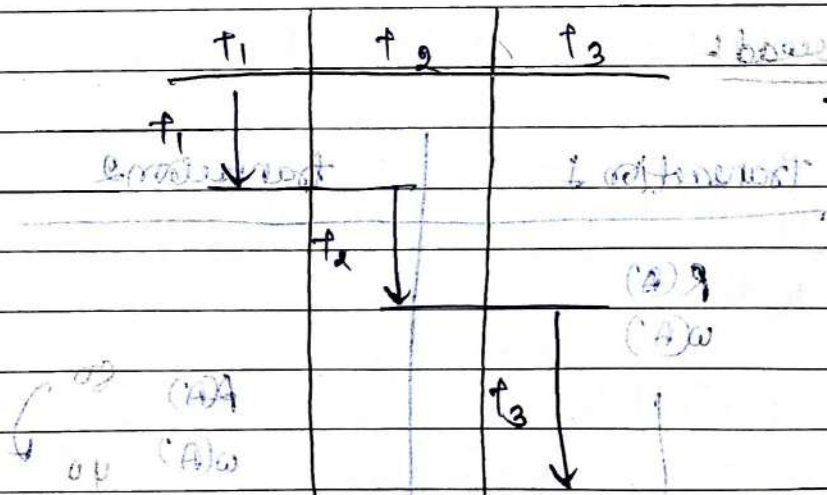
① Serializability helps in Identifying which Non-serial schedule gives consistent result as given by serial schedule.

Schedule:-

It is chronological execution sequence of multiple transactions.

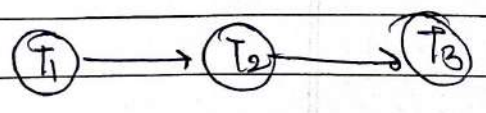
→ Types $\begin{cases} \rightarrow \text{Serial.} \\ \rightarrow \text{Parallel.} \end{cases}$

• Serial:- (Concurrent)



① first complete then Second start.

→ Give concurrent result.
→ Consistent.
→ Less throughput.

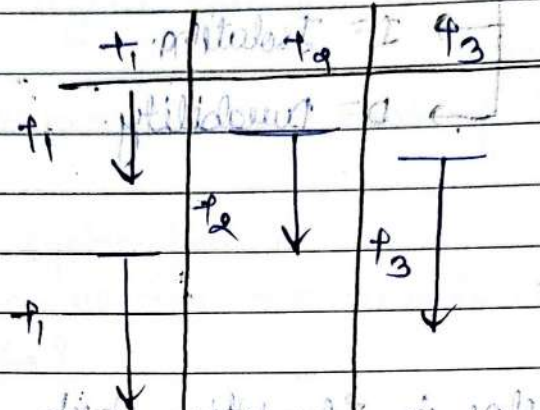


→ preserve database consistency.

→ Time complexity (waiting):

Ex:- ATM.

• Parallel schedules (concurrency)



Ex:- Bank app. making is these transactions and more throughput.

All concurrency problem.

1. Dirty read.
2. Incorrect summary.
3. Lost update.
4. Unrepeatable read.
5. Phantom read.

① Dirty read:

with database temp

tests cases

Transaction 1

Transaction 2

100 R(A)
80 W(A)

AA) 80
W(A) 40

commit

Failure

rollback

⑥ conflict:

1. $I_i = \text{read}(Q)$ $I_j = \text{read}(Q)$ non-conflict.
2. $I_i = \text{read}(Q)$ $I_j = \text{write}(Q)$ Then conflict. R-W
3. $I_i = \text{write}(Q)$ $I_j = \text{read}(Q)$ Then conflict. W-R
4. $I_i = \text{write}(Q)$ $I_j = \text{write}(Q)$ Then conflict. W-W

⑦ If a schedule S can be transformed into a schedule S' by a series of swaps of non-conflict instructions, we say S and S' are conflict equivalent.

T_1	T_2	T_1	T_2
$R(X)$			$W(Y)$
	$W(Y)$	$R(X)$	
$W(X)$		$R(X)$	
Non-serial		Serial.	

Ques:

check whether the given schedule is conflict serializable or not.

$S: R_1(A), R_2(A), R_1(B), R_2(B), R_3(B), W_1(A), W_2(B)$

Answer

T_1	T_2	T_3
$R_1(A)$		
	$R_2(A)$	
$R_1(B)$		
	$R_2(B)$	
		$R_3(B)$
$W_1(A)$		
	$W_2(B)$	

Conflict Equivalence

(i)

S	
T ₁	T ₂
R(A)	
W(A)	
	R(A)
	W(A)
R(B)	

S'	
T ₁	T ₂
R(A)	
W(A)	
R(B)	
	R(A)
	W(A)

We can swap the non-conflict pairs.

• Non-conflict pair

- (i) R(A) R(A)
- (ii) R(B) R(A)
- (iii) R(B) W(A)
- (iv) W(B) R(A)
- (v) W(A) W(B)

• conflict pair

- (i) R(A) W(A)
- (ii) W(A) R(A)
- (iii) W(A) W(A)

(ii)

S	
T ₁	T ₂
R(A)	
W(A)	
W(A)	R(A)
R(B)	

We can't swap it because of conflict pair.

Note:-

S → S'

So, S' is serializable

Precedence graph

Date / /

Page No.

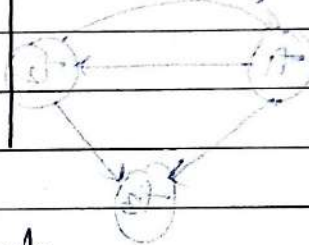
Shivatal

Ques ①

check conflict pairs in other transactions and draw edge.

T_1	T_2	T_3	
$R(x)$		$R(y)$	$R(x)$
		$R(x)$ (new)	$R(x)$
	$R(y)$		$R(x)$
	$R(z)$ (new)		$R(x)$
		$w(y)$	
	$w(z)$		
$R(z)$			
$w(x)$			
$w(z)$			

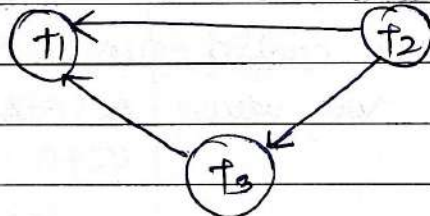
doing matching



$\therefore w(y) \leftarrow R(y)$
 $\therefore w(y) \leftarrow w(y)$

→ first make precedence graph

(new ...)



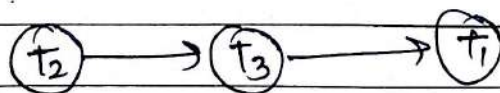
(Conflict)

→ check loop / cycle

$\therefore$ No loop / No cycle

So, it is a conflict serializable schedule.
and consistent.

→ check Indegree = 0.

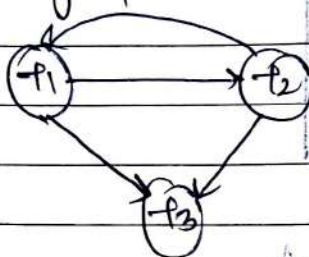


① View serializable

②

S			
T ₁	T ₂	T ₃	
R(A)			(x)
	W(A)		
W(A)			
		W(A)	

③ precedence graph



(non-conflict)

→ Non-conflict serializable (∵ loop)

If S and S' are conflict equivalent then it will be view serializable. vice-versa is not true

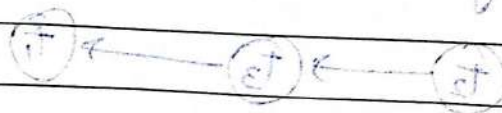


Why / goal

Why all / goal

Subsets relations, bijection is full p2
 to be true

∴ conflict starts



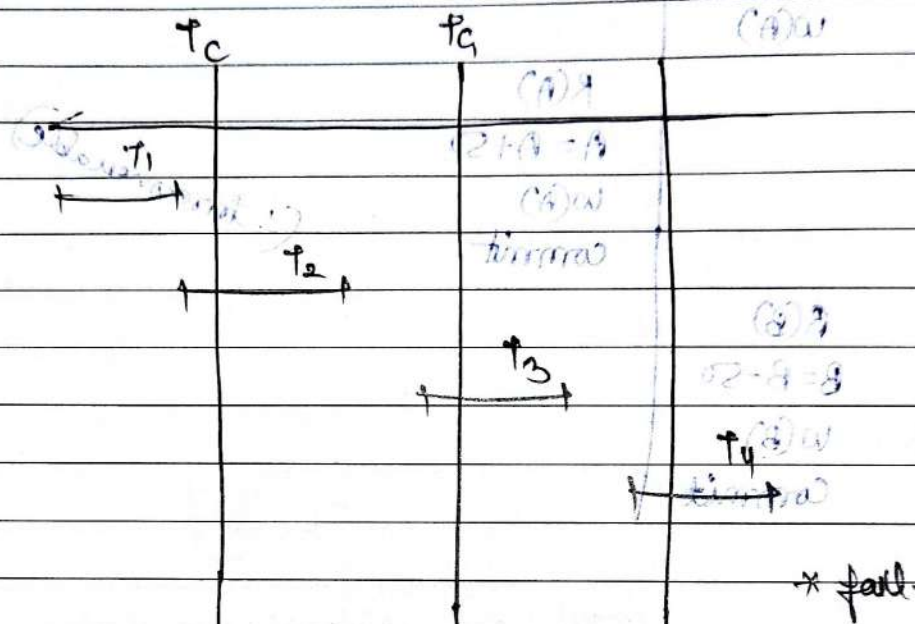
① Recoverable schedule:- and Unrecoverable schedule:-

Ex	T ₁	T ₂	Result
	R(A)		
	A = A + 50		
	W(A)		
		R(A)	
		A = A + 50	
		W(A)	
		Commit	
	R(B)		
	B = B - 50		
* fail	W(B) =		
	Commit		

tx.	T_1	T_2
	$R(A)$	
	$A = A + 50$	
	$W(A)$	
		$R(A)$ (→ recoverable)
		$A = A + 50$
		$W(A)$
	$R(B)$	
	$B = B - 50$	
	$co(B)$	
* fail	commit	

① Checkpoint:

② Redoing / undoing all transactions succeeded in the log can be very slow.



checkpoint

$T_4 \rightarrow$ undo

$T_3 \rightarrow$ Redo

$T_2 \rightarrow$ Redo

$027A = A$

(A)W

(A)W

$027A = A$

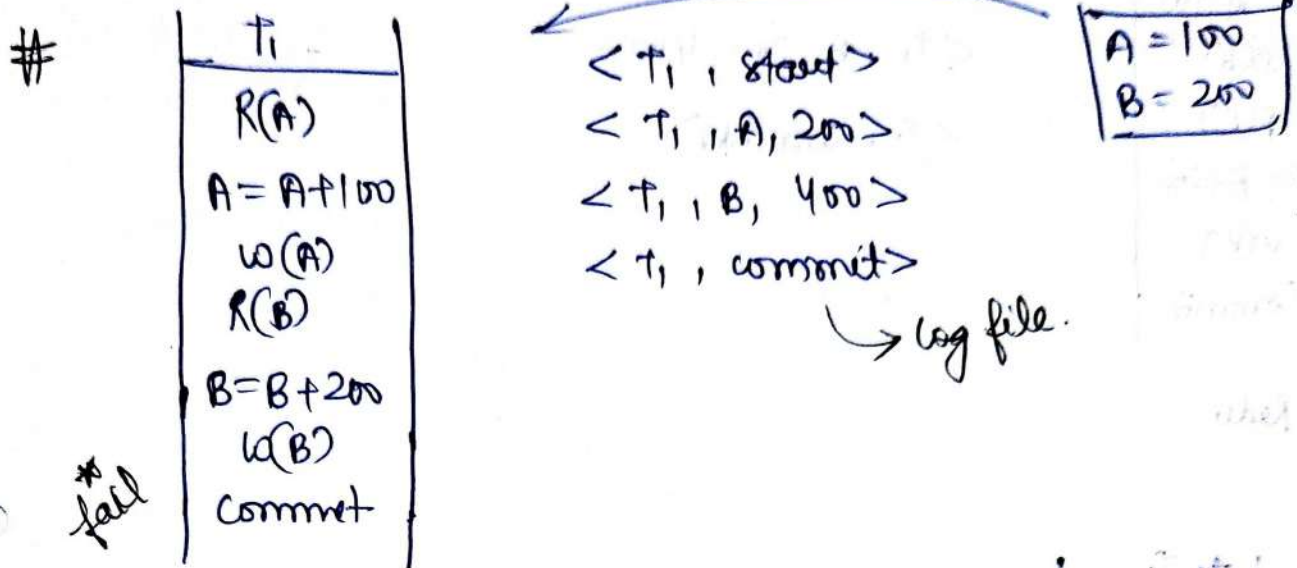
(A)W

$027A = A$

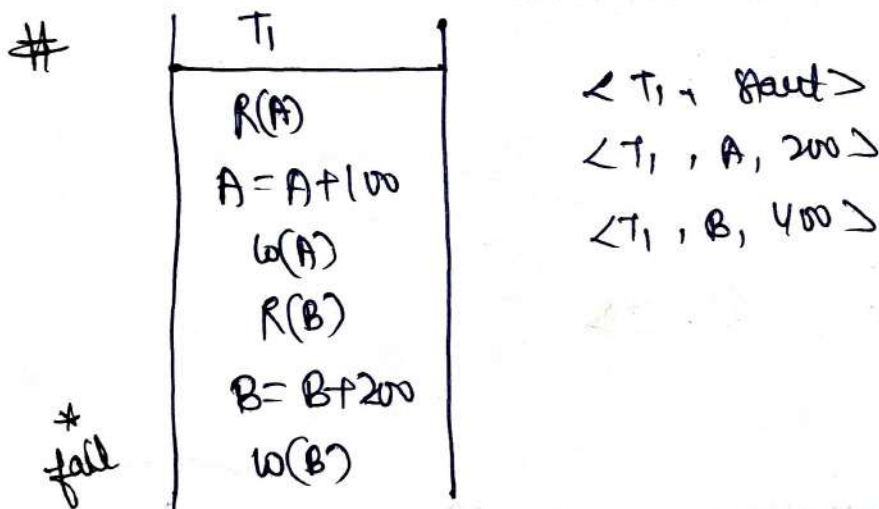
(A)W

① log based recovery:-

(i) Deferred Database Modification:- (Redo/No-undo)



② Redo / No undo

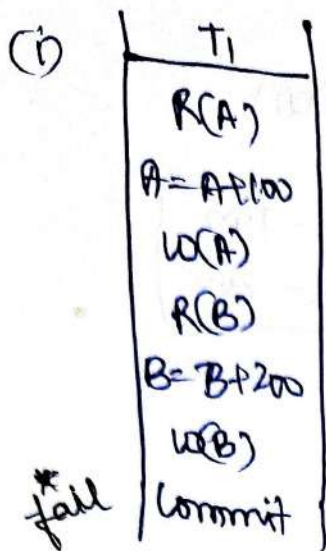


③ Rollback.

(ii) Immediate Database modification:- (Redo/undo)

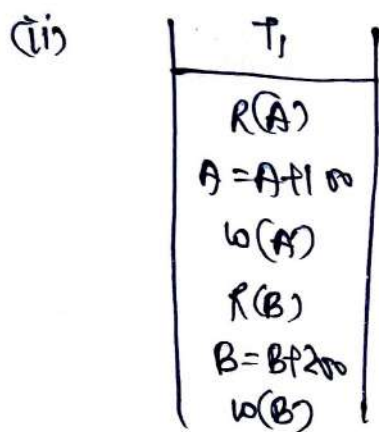
A = 100
B = 200

→ The change will be save along with.



$\langle T_1, \text{Start} \rangle$
 $\langle T_1, A, 100, 200 \rangle$
 $\langle T_1, B, 200, 400 \rangle$
 $\langle T_1, \text{Commit} \rangle$

redo

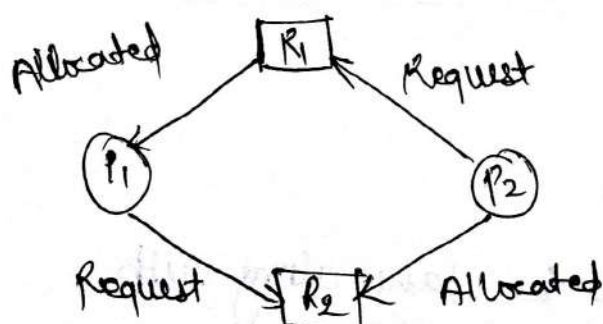


$\langle T_1, \text{Start} \rangle$
 $\langle T_1, A, 100, 200 \rangle$
 $\langle T_1, B, 200, 400 \rangle$

undo / rollback.

Deadlock:-

If two or more processes are waiting on happening of some event, which never happens, then we say these processes are involved in deadlock then that state is called deadlock state.



→ Condition for deadlock

- (i) Mutual Exclusive.
- (ii) No preemption.
- (iii) Hold & wait
- (iv) Circular wait.

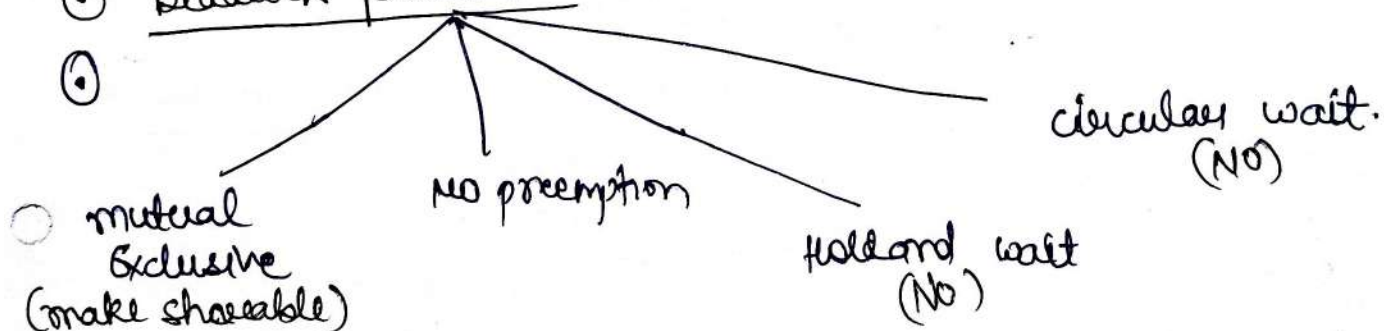
① Methods to handle deadlocks:-

1. Deadlock Ignorance (ostrich method).
2. Deadlock prevention.
3. Deadlock detection & recovery
- ① 4. Deadlock Avoidance (Banker's Algorithm).

① Deadlock Ignorance:-

- ① Because of that do not want affect the performance and speed. Ex - windows.

① Deadlock prevention:-



Make any one or all four we make false to prevention.

① Deadlock detection & Recovery:-

- ① Detect, kill the processes or process.

① concurrency control techniques

② locking techniques for concurrency control:-

shared-exclusive locking:-

① shared lock (S) = If transaction locked data item in shared mode then allowed to read only.

② Exclusive lock (X) = If transaction locked data item in exclusive mode then allowed to read and write both.

1. Shared lock (S):-

T ₁	
S(A)	shared lock
R(A)	
U(A)	unlock

2. Exclusive lock:-

T ₁	
X(A)	Exclusive lock.
R(A)	
W(A)	
U(A)	unlock.

Request

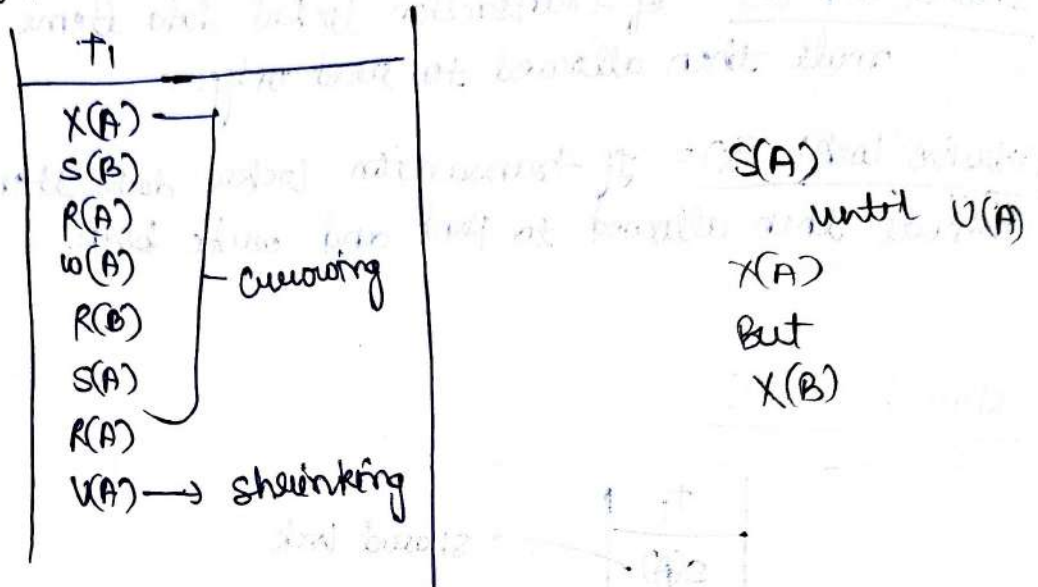
grant

	S	X
S	Yes	No
X	No	No

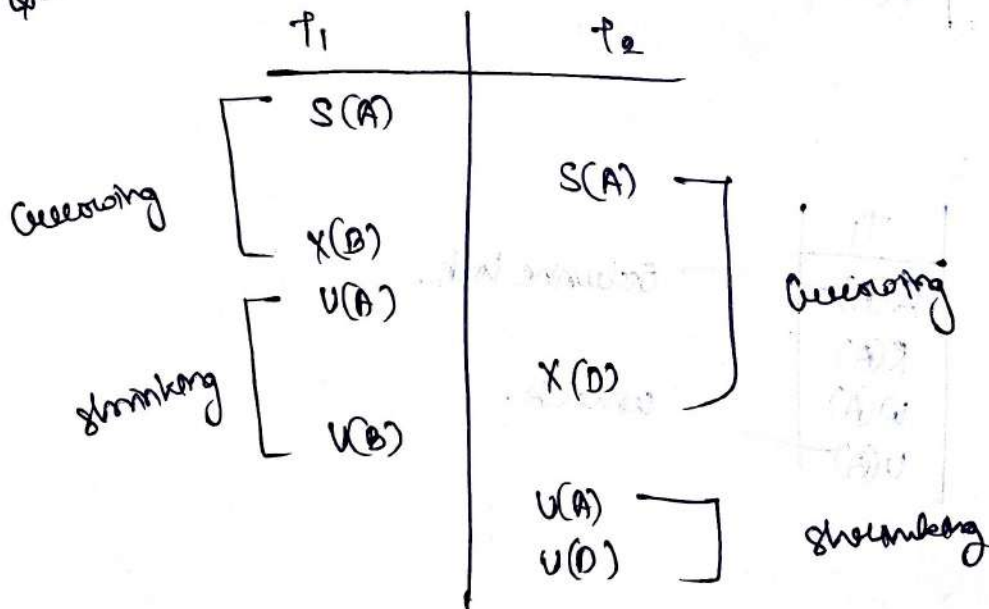
① 2-phase locking (2PL):-

Growing phase:- locks are acquired and no locks are released.

Shrinking phase:- locks are released and no locks are acquired.



\$-



Serializable

① → ② (lock point, first).

⑥ Advantages

(1) Always ensure serializability.

⑦ Disadvantages

(i) May not free from irrecoverability.

T_1	T_2
$X(A)$	
$R(A)$	
$W(A)$	
$U(A)$	
$A(A)$	$S(A)$
*fail	$R(A)$
	commit

(because of commit).

(ii) Not free from cascading rollback.

T_1	T_2	T_3
$X(A)$		
$R(A)$		
$W(A)$		
$U(A)$		
	$S(A)$	
	$R(A)$	
		$S(A)$
		$R(A)$

Roll back (pointing to T_1)

*fail (pointing to T_1)

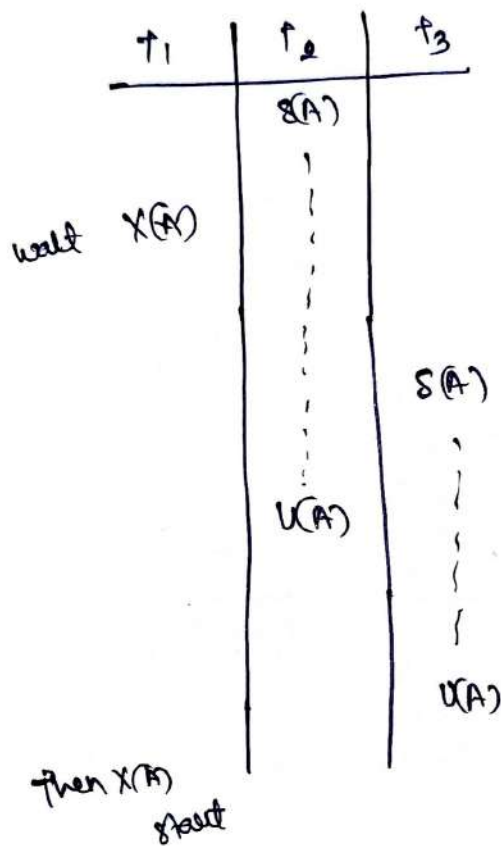
Rollback (pointing to T_3)

(all rollback due to fail in T_1)

(iii) Not free from deadlock.

	τ_1	τ_2	
arriving	$X(A)$	$X(B)$ arriving	(waiting)
wait	$X(B)$	$X(A)$ wait	

(iv) Not free from starvation.



(τ_1 is waiting due to τ_2 then τ_3 and so on)

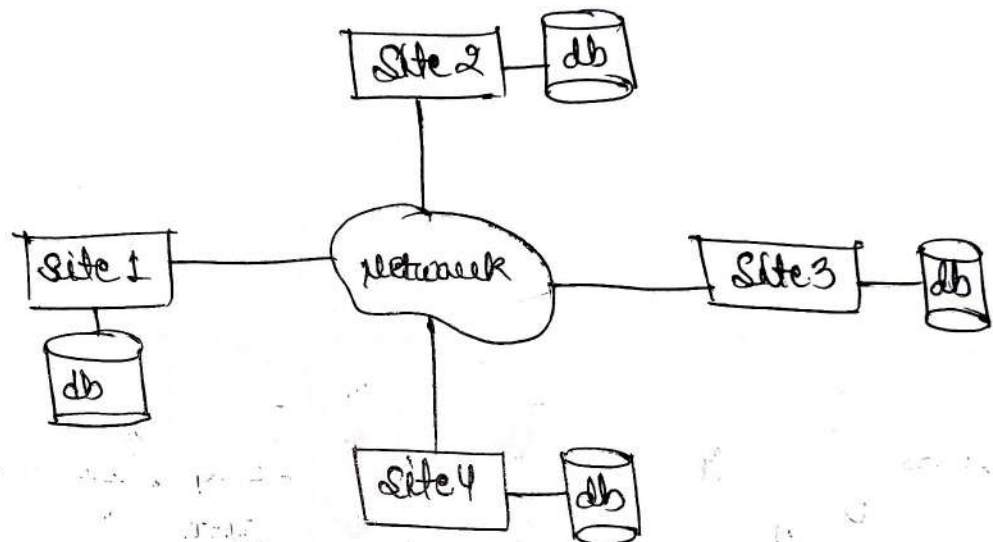
($\therefore$ waiting [$X(A)$])

① Distributed Database:-

Introduction to Distributed Database:-

② DDB is a collection of multiple, logically interrelated databases distributed over a computer N/w.

DDDBMS:- It is a software system that manages a distributed database while making distribution transparent to user.



Site = computer in distributed system.
db = database

∴ Distributed database system.

1. Distributed database system:-

- (i) Database is geographically separated.
- (ii) There may be homo / heterogeneity of sites.

VS

(ii) Parallel database system:-

- (i) on multiprocess and single database system
- (ii) symmetry and homogeneity of sites.

Features of distributed database:-

- (i) Data is stored in a no. of sites.
- (ii) sites are interconnected by a Network.
- (iii) DDB is logically a single database.
- (iv) DDBMS has full functionality of DBMS.

Advantages of distributed database:-

1. sharing of data

2. Improved availability and reliability.

System is running continuously

System is running at any given point of time.

→ failure of a single site does not affect the entire system.

3. Autonomy:- It implies that the operations at a given sites are controlled by that site.

4. Easier expansion:- Any other site can be add easily without any problem.

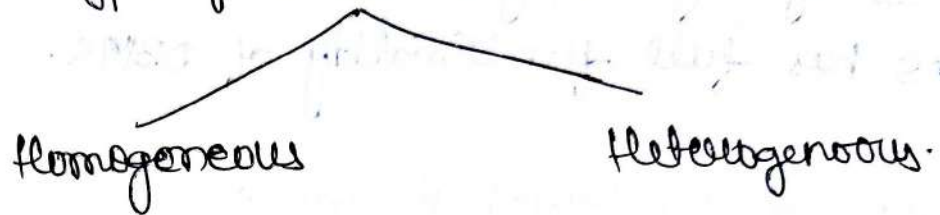
5. Reduce operating cost

6. fast data processing

Disadvantages of distributed database:-

1. Complexity of management and control:
2. Difficulty in deadlock handling.
3. Security.
4. Lack of standards.

Types of distributed Database:-

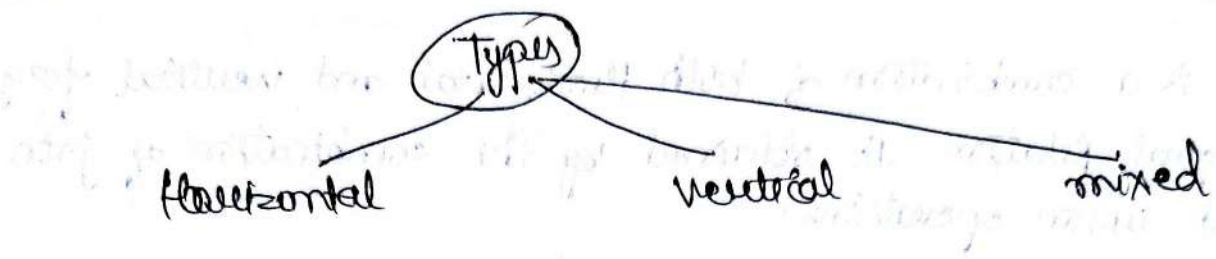


1. Homogeneous:-

1. Share a common Global schema.
2. Run identical DBMS system.
3. Each sites provides part of its autonomy in terms of write to change schema or software.
4. Easy transaction processing

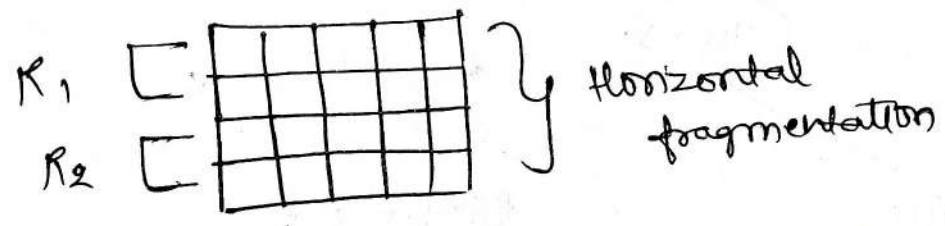
① Data Fragmentation

- ① It is the system partitions / divides the relation into several fragments, and stores each fragment at a different site.



1. Horizontal

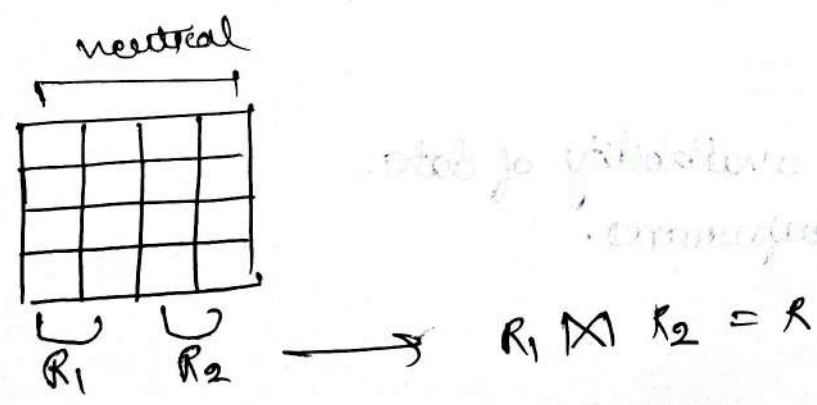
- ① It breaks relation 'R' by assigning each tuple of R to one or more fragments.
- ② Each fragment is a subset of tuple in original reln 'R'.



$R_1 \cup R_2 = R$ (To re-construct it we use union ($\cup$)).

2. Vertical fragmentation

- ① It breaks relation 'R' by decomposing schema.
- ② Each fragment is a subset of the attribute of the original reln 'R'.



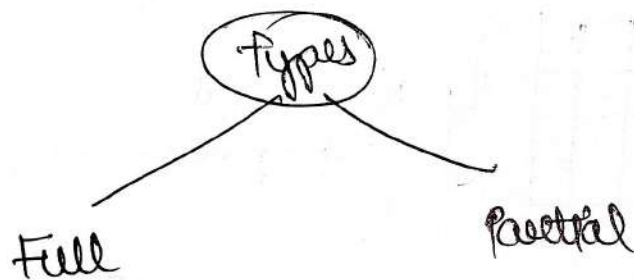
- ⑥ To reconstruct it we use Natural join, and divide the fragmentation in a way that there is some common in different fragmentation to apply natural join.

(iii) Mixed

- ⑥ It is a combination of both horizontal and vertical fragments. original relation is obtained by the combination of join and union operations.

⑥ Replication

- ⑥ It means storing a copy (all replica) of a relation or relation fragments in two or more sites.



(i) Full Replication

- ⑥ Distribution of entire relation at all the sites.

(ii) Partial Replication

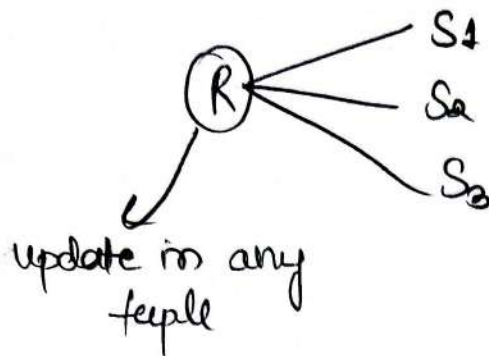
- ⑥ Only some fragments of a relation are replicated.

⑥ Advantages

- (i) Increase availability of data.
- (ii) Better performance.

⑥ Disadvantages

1. Updating



→ overhead increase in case of update operation.

⑥ Transparency

⑥ In distributed system the user should be able to access the database exactly as if the system were local. Hiding details such as data storage, how data can be accessed is called as data transparency.

- (i) Location transparency.
- (ii) Fragmentation transparency.
- (iii) Replication transparency.
- (iv) Naming transparency.